Acropolis Institute of Technology and Research, Indore

Department of Computer Science and Engineering

B. Tech. IV Semester

Jan - June 2026

Lab Assignment

On

Software Engineering [CS 403]

Submitted To:

Prof Sushma Khatri

Senior Assistant Professor

Submitted By:

Naman Nekiye

(0827CS241159)

Course Educational Objectives

CEO1	Understand the basics of software development life cycle as a product.
CEO2	Understand the current requirements of industries for software standards
CEO3	Implement the software as a product using different design patterns
CEO4	Apply the software development techniques in real life applications.
CEO5	Understand the existing software solutions and correlate with the SDLC, design patterns and software standards

Course Outcomes

Upon completion of this subject / course the student will be able:

CO1	Specify, classify, implement, analyze and develop applications using various SDLC models like Linear sequential, prototype, Evolutionary models
CO2	Understand, classify, analyze and develop applications using Various software standards. .
CO3	Understand, classify, implement various types of design patterns (Creational, Behavioural, structural); understand and analyze each design pattern using java program
CO4	Understand, classify, implement various types of design patterns (Presentation layer, Integration layer, business layer); Analyze the problem statement make UML diagram and code the program in java .
CO5	Able to describe the SDLC, design patterns and software standards applicable to the existing software systems.

TABLE OF CONTENTS

S. NO.	TOPIC	PAGE NO	Date of Experiment	Date of Submission	REMARK
1.	EXPLAIN SDLC IN DETAIL. (HANDWRITTEN)				
2.	COMPARE FOLLOWING MODELS: (HANDWRITTEN) A. LINEARSEQUENTIAL MODEL, B. PROTOTYPING MODEL, C. RAD MODEL, EVOLUTIONARY PROCESS MODELS LIKE D. INCREMENTAL MODEL, E. SPIRAL MODEL, F. COMPONENT ASSEMBLY MODEL, G. RUP AND H. AGILEPROCESSES I. CMM POINT FOR COMPARISON: A. YEAR B. PROPOSED BY C. DIAGRAM D. BRIEF DESCRIPTION E. ADVANTAGES F. DISADVANTAGES G. WHEN BEST SUITED				
3.	PROBLEM STATEMENT: IDENTIFYING THE REQUIREMENTS				
4.	MODELING UML USE CASE DIAGRAMS				
5.	SYNOPSIS				
6.	ESTIMATION OF PROJECT METRICS				
7.	MODELING DATA FLOW DIAGRAMS				

8.	E-R MODELING FROM THE PROBLEM STATEMENTS				
9.	IDENTIFYING DOMAIN CLASSES FROM THE PROBLEM STATEMENTS				
10.	MODELING UML CLASS DIAGRAMS				
11.	MODELING UML SEQUENCE DIAGRAMS				
12.	STATE TRANSITION DIAGRAM				
13.	ACTIVITY MODELING				
14.	ESTIMATION OF TEST COVERAGE METRICS AND STRUCTURAL COMPLEXITY				
15.	DESIGNING TEST SUITES				
16.	SOFTWARE REQUIREMENT SPECIFICATION (SRS)				

Experiment I

EXPLAIN SDLC IN DETAIL (HANDWRITTEN)

Experiment - I

Q)) Explain SDLC in detail.

S⇒ Software development Life Cycle (SDLC) is a structured process used by Software engineers to design, develop, test, and maintain high-quality Software systems. It provides a systematic approach to software development so that the final product meets user requirements within time and budget constraints. Without SDLC, Software development would be unorganized, leading to poor quality, delays, and increased costs. SDLC ensures that every stage of development is carefully planned and executed reducing risks and improving efficiency.

→ In modern software engineering, SDLC acts as a backbone for all types of projects, whether small applications or large enterprise systems. It defines a clear sequence of steps, starting from understanding user needs to maintaining the final product after deployment. Each Phase has its own importance, and skipping any Phase can negatively affect the overall system.

④ Phases of SDLC:→

- **Requirement Gathering and Analysis:→**

→ The SDLC begins with the requirement gathering and analysis phase, which is considered the most crucial part of the entire process. In this phase, developers, analysts, and stakeholders interact with clients to understand their needs and expectations from the software. All functional requirements, such as features and operations, as well as non-functional requirements like performance, security, and usability, are carefully identified.

→ A feasibility study is also conducted to determine whether the project is technically, economically, and operationally viable. Based on all the collected information, a document known as the Software Requirement Specification (SRS) is prepared. This document serves as a reference for all future stages of development. If the requirements are not properly understood at this stage, it can lead to major issues later in the project.

- System Design :→

→ once the requirements are clearly defined, the next Phase is System design. In this Phase, the overall structure and architecture of the Software System are Planned. The design Phase transforms the requirements into a blueprint that developers can follow during coding.

→ The design process is usually divided into High-Level Design (HLD) and Low-Level Design (LLD). High-Level Design focuses on the overall system architecture, including modules, data flow, and system components. Low-Level Design, on the other hand, deals with detailed design aspects such as algorithms, data structures, and individual modules. The design Phase also includes database design, user interface design, and system flow diagrams. A well-prepared design ensures that the development process becomes smoother and more efficient.

- Implementation (Coding) :→ The implementation Phase is where the actual development of the software takes place. Based on the design documents, developers write code using programming languages such as C, C++, Java or Python.

The System is usually divided into Smaller modules, and each module is developed separately to make the process manageable.

→ During this Phase, developers follow Coding standards and guidelines to ensure Code quality and readability. Regular reviews and debugging are performed to identify and fix errors at an early stage. The implementation Phase Converts the theoretical design into a working software system. The efficiency and quality of Coding directly impact the performance of the final product.

- Testing :→

→ After the Coding Phase, the software enters the testing Phase, where it is checked for errors and defects. Testing ensures that the software functions Correctly and meets all the specified requirements. It is one of the most important Phases because even Small errors can lead to major failures in real-world applications.

→ Different types of testing are performed, including unit testing, where individual modules are tested; integration testing,

where combined modules are tested; system testing, where the entire system is evaluated; and acceptance testing, where the software is validated by the client. The goal of testing is ~~so~~ to ensure that the software is reliable, secure, and free from bugs before it is delivered to users.

- **Deployment:** →

→ Once the software passes all testing stages, it is ready for deployment. In this phase, the software is released to the users or installed in the production environment. Deployment can be done in different ways, such as full deployment or gradual deployment in phases.

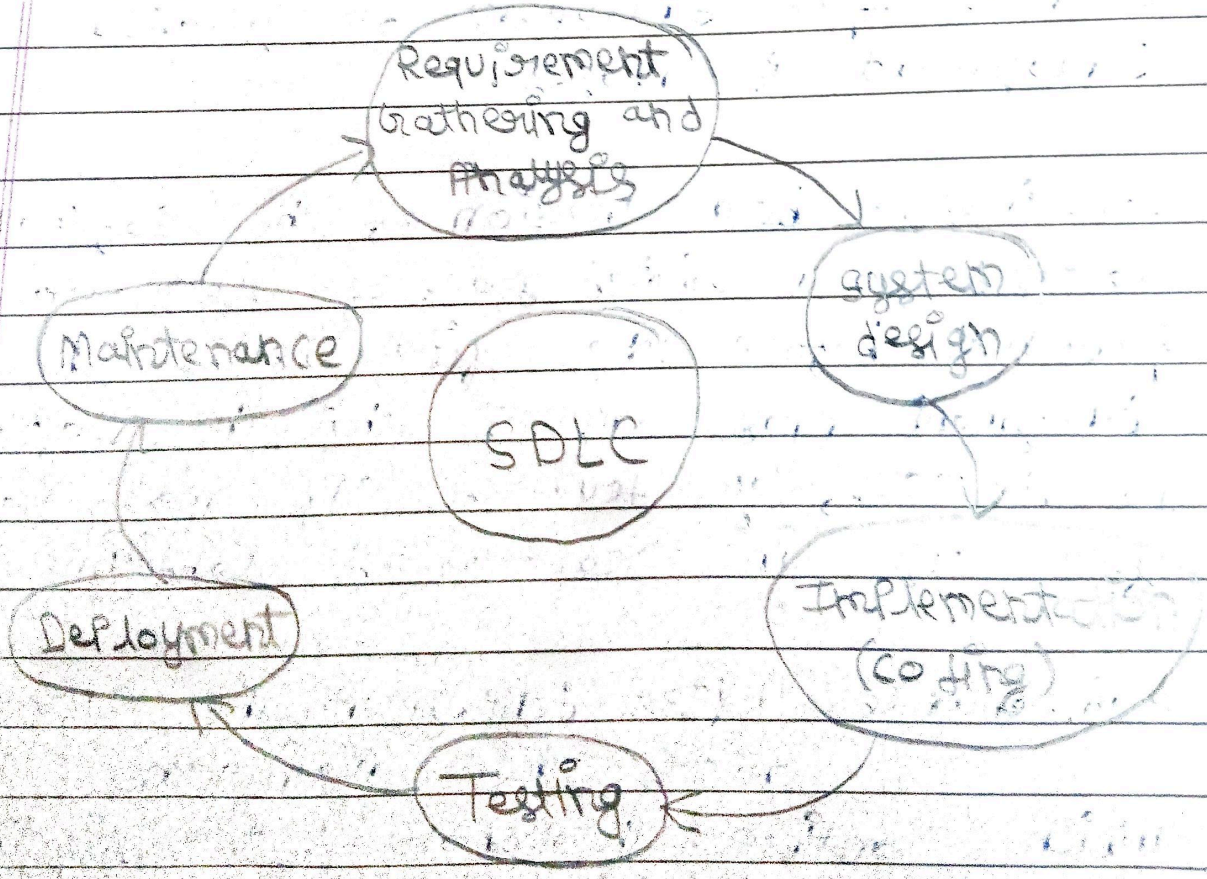
→ Sometimes, a beta version of the software is released to a limited group of users to gather feedback before the final release. The deployment phase ensures that the software is successfully delivered and made available for use in real-world conditions.

- **Maintenance:** → After deployment, the software enters the maintenance phase, which continue throughout its life cycle. In this phase, developers monitor the system and fix any issues that arise

during actual usage. Maintenance also includes updating the Software to adapt to new technologies, improve performance, or add new features.

→ There are different types of maintenance, such as Corrective maintenance for fixing bugs, adaptive maintenance for handling environmental changes, and Perfective maintenance for enhancing system performance. This Phase ensures that the software remains functional and relevant over time.

→ SDLC Cycle →



- **SDLC Models:-**

→ SDLC can be implemented using different models depending on Project requirements. The Waterfall model follows a linear approach where each Phase is completed before moving to the next. The spiral model focuses on risk analysis and iterative development. The Incremental model develops the Software in small parts, while Agile emphasizes flexibility and continuous feedback. Each model has its own advantages and is chosen based on the nature of the Project.

- **Advantages of SDLC:-**

→ SDLC Provides a structured approach to Software development, making it easier to Plan and manage Projects. It improves the quality ~~the~~ of Software by ensuring that each Phase is properly executed. It also reduces development cost and time by identifying issues early in the Process. Moreover, SDLC enhances communication among team members and ensures better control over the Project.

DATE / /
PAGE 9

- **Disadvantages of SDLC:→**

→ Despite its benefits, SDLC also has some limitations. It can be time consuming, especially for large projects, as each phase requires detailed documentation. Traditional models like Waterfall lack flexibility and may not adapt well to changing requirements. Additionally, improper execution of any phase can affect the entire development process..

Experiment II

COMPARE FOLLOWING MODELS: (HANDWRITTEN)

1. **LINEARSE#QUENTIAL MODEL,**
2. **PROTOTYPING MODEL,**
3. **RAD MODEL,**

EVOLUTIONARY PROCESS MODELS LIKE

4. **INCREMENTAL MODEL,**
5. **SPIRAL MODEL,**
6. **COMPONENT ASSEMBLY MODEL,**
7. **RUP AND**
8. **AGILEPROCESSES**
9. **CMM**

POINT FOR COMPARISON:

A. YEAR

B. PROPOSED BY

C. DIAGRAM

D. BRIEF DESCRIPTION

E. ADVANTAGES

F. DISADVANTAGES

G. WHEN BEST SUITED

DATE / /
PAGE 231

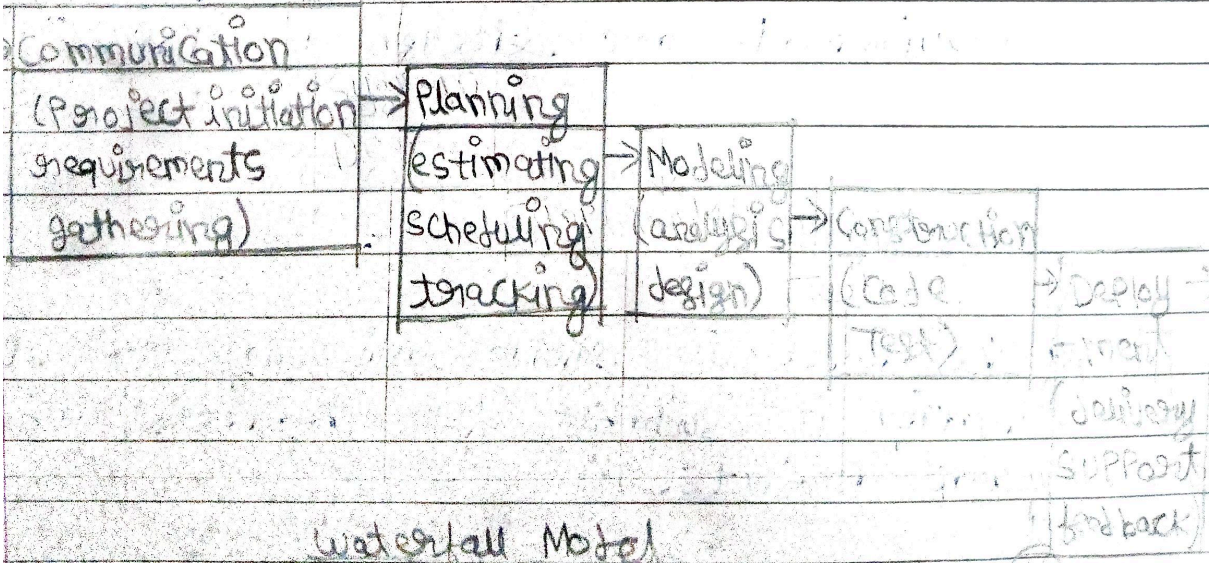
Experiment - II

Linear Sequential Model (Waterfall Model) :→

Year ⇒ The linear sequential model was formally introduced in the year 1970. It is considered one of the earliest structured approaches to software development and became widely used in the initial stages of software ~~development~~ engineering as a discipline.

Proposed By ⇒ This model was proposed by Winston Royce, an American computer scientist. Although he originally discussed its limitations, the model later became popular as a simple and systematic approach to software development.

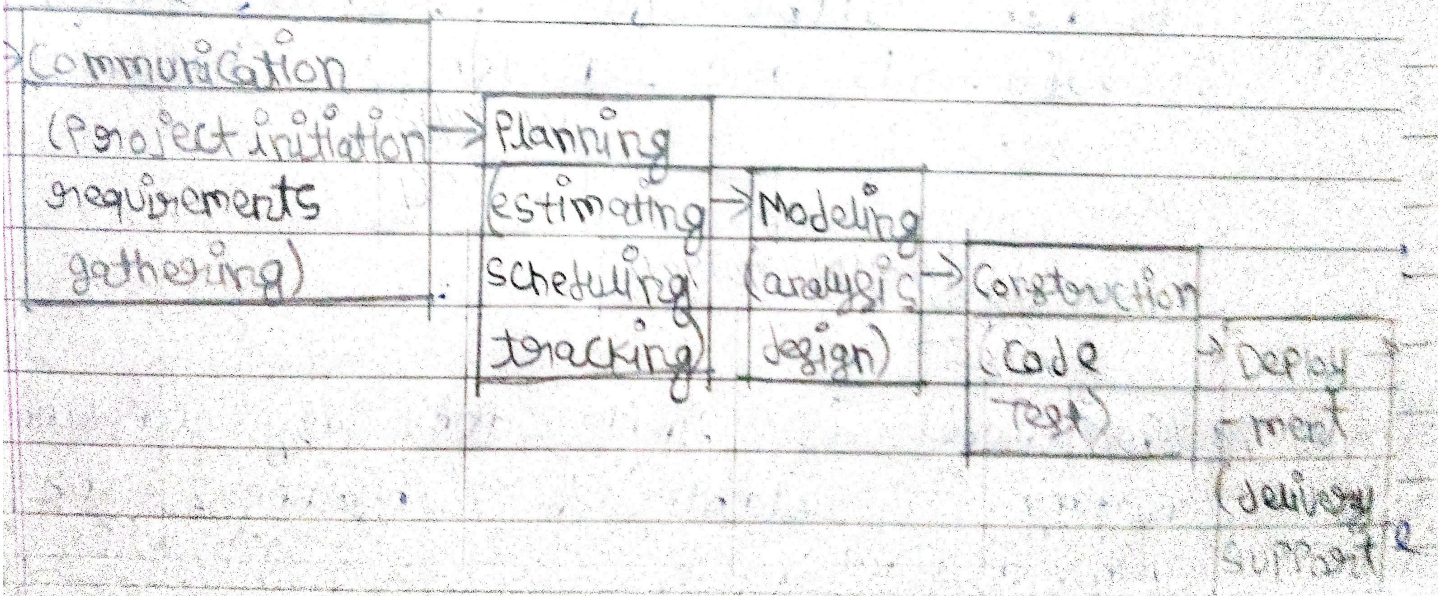
Diagram ⇒



Waterfall Model

Proposed By ⇒ This model was proposed by Winston Royce an American Computer scientist. Although he originally discussed its limitations, the model later became popular as a simple and systematic approach to software development.

Diagram ⇒



- **Brief Description:→**

→ The linear sequential Model, also known as the waterfall model, is a traditional software development approach in which the development process flows in a straight, sequential manner. Each Phase must be completed before the next Phase begins, and there is no overlap between Phases. The major stages include requirement analysis, system design, coding, testing, deployment and maintenance.

→ This model follows a very rigid structure, meaning that once a Phase is completed, it is difficult to return and make changes. Therefore, all requirements must be clearly defined at the beginning of the Project. It focuses heavily on documentation and Planning making it easier to manage and understand. However, due to its lack of flexibility, it is not suitable for Projects where requirements are expected to change frequently.

Advantages:→

The model is simple and easy to understand making it suitable for beginners and small Projects.

② It follows a structured and easy to understand disciplined approach, where each phase has clearly defined outputs and documentation.

③ Since everything is planned in advance, project management becomes easier and more organized.

④ Proper documentation at every stage helps in future maintenance and understanding of the system.

- Disadvantages :-

① The model is not flexible, and making changes after a phase is completed is very difficult and costly.

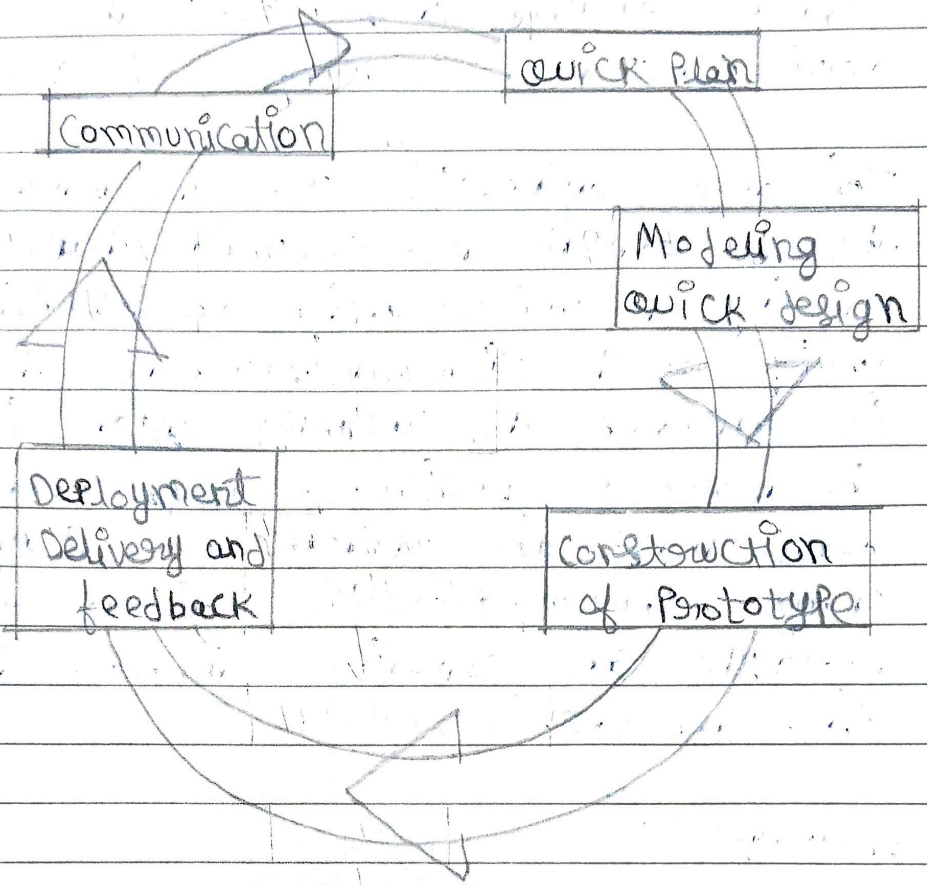
② Errors are usually detected at a later stage during testing, which can increase development time.

③ It is not suitable for complex or long-term projects where requirements may evolve over time.

④ Lack of user interaction during development can lead to a final product that does not fully meet user expectations.

DATE / /
PAGE # 811

Diagram :->



Prototyping Model

Brief Description :->

The Prototyping Model is a software development approach in which a preliminary version of the software, called a prototype, is built to understand user requirements more clearly. Instead of gathering all requirements at once, developers create a working model of the

System and present it to users for feedback. Based on this feedback, the prototype is refined multiple times until the final requirements are clearly understood.

→ This model emphasizes user involvement throughout the development process, which helps in reducing misunderstanding and improving the quality of the final product. once the prototype is approved, the actual system is developed. The model is particularly useful when requirements are unclear or likely to change. However, excessive reliance on prototype can sometimes increase development time and cost.

- Advantages :→

① Helps in better understanding of user requirements through interaction and feedback.

② Reduces the chances of requirement-related errors in the final system.

③ Users get an early view of the system, which increases satisfaction and confidence.

④ Improves communication between developers and clients.

- Disadvantages:-

(1) Developing multiple prototypes can increase cost and time.

(2) Users may expect the prototypes to be the final product, leading to confusion.

(3) Users ~~m~~ frequent changes can make the system design complex and unstructured.

(4) Not suitable for very large or highly complex system.

- When best suited:-

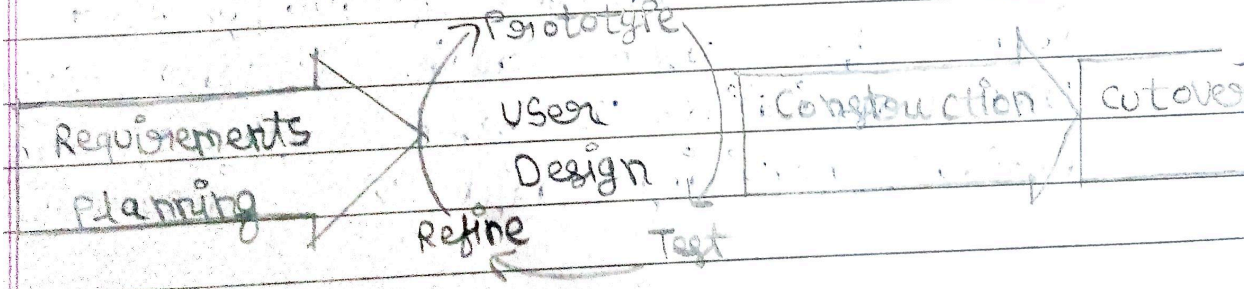
→ The Prototyping Model is best suited for projects where requirements are not clearly defined at the beginning or are expected to change frequently. It is ideal for user-centric systems such as web applications, UI-based software, and projects requiring high user interaction.

RAD Model (Rapid Application Development) :-)

Year :-) The RAD (Rapid Application ~~Model~~ Development) Model was introduced in the 1980s, with major development and popularity around 1991. It emerged as a response to the slow and rigid nature of traditional models like Waterfall.

Proposed By :-) The RAD Model was proposed by James Martin, a British information technology Consultant, who emphasized faster software development using reusable components and user involvement.

Diagram :-)

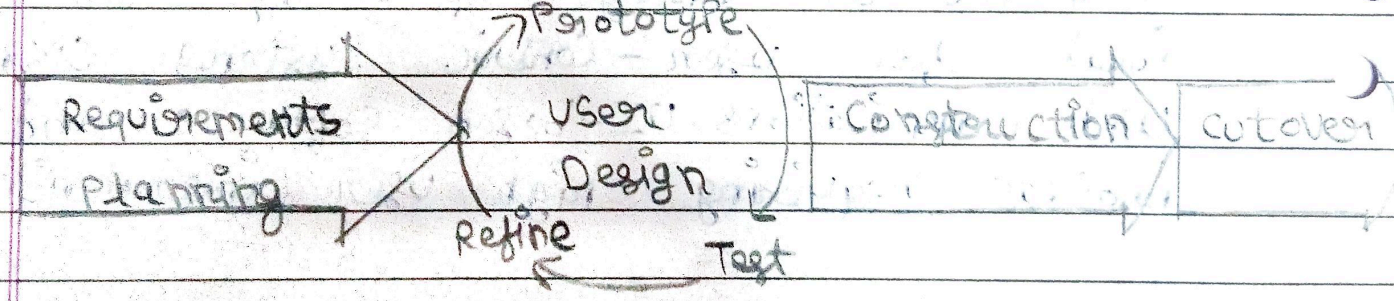


RAD Model

- **Year:-)** The RAD (Rapid Application Development) Model was introduced in the 1980s, with major development and popularity around 1991. It emerged as a response to the slow and rigid nature of traditional models like Waterfall.

- **Proposed By:-)** The RAD Model was proposed by James Martin, a British information technology Consultant, who emphasized faster Software development using reusable components and user involvement.

- **Diagram:-)**



RAD Model

- Brief Description:->

→ The Rapid Application Development (RAD) Model is an iterative and incremental Software development approach that focuses on fast development and quick delivery of Software. Unlike traditional models, RAD emphasizes building Software in a short time using reusable components, prototyping, and active user participation.

→ In this model, development is divided into smaller modules that can be developed in Parallel. Continuous feedback is taken from users during the design and development stages, allowing a quick modifications and improvements. The RAD Model reduces development time significantly and increases Productivity. However, it requires skilled developers and may not be suitable for large-scale systems where high performance and scalability are critical.

- Advantages:->

(1) Enables very fast development and delivery of Software.

(2) Encourages user involvement, leading to better requirement understanding.

③ Reusable components reduce development effort and time.

④ Flexible and allows quick changes during development.

- Disadvantages :->

① Requires highly skilled and experienced developers.

② Not suitable for very large or complex systems.

③ Depends heavily on user availability for continuous feedback.

④ May lead to poor design if speed is prioritized over quality.

- When Best Suited :->

→ The RAD Model is best suited for projects that require quick development and delivery, where requirements are fairly well understood but may need refinement. It is ideal for small to medium-sized applications, especially those with tight deadlines and active user development.

DATE: / /
PAGE: 17

Incremental Model:→

Years:→ The incremental model was introduced in the 1970s and became more widely used in the 1980s as developers started moving away from rigid models like waterfall.

Proposed By:→ The incremental model does not have a single specific inventor. It evolved as an improvement over earlier models, combining elements of the waterfall model with iterative development practices.

Diagram:→

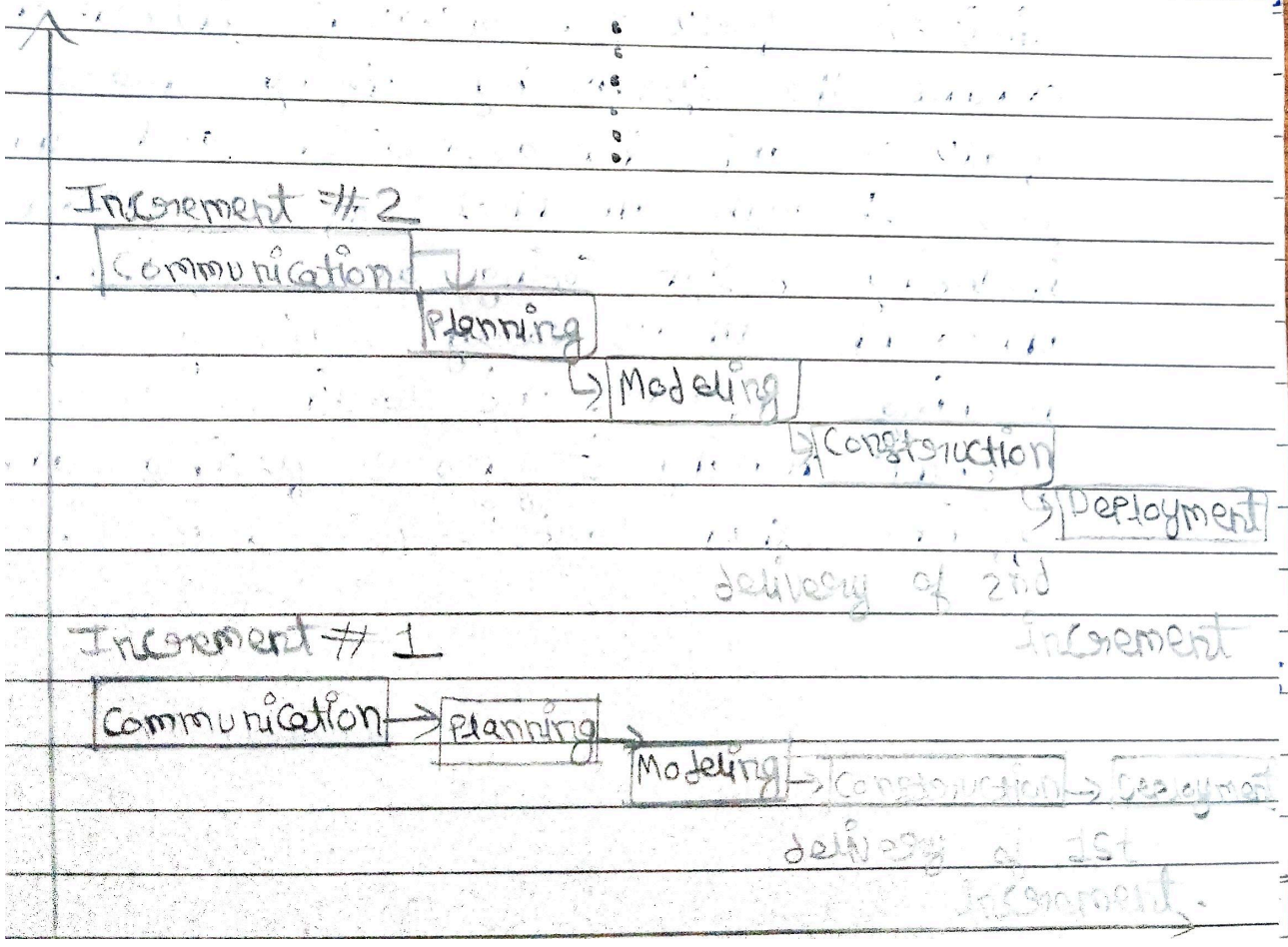
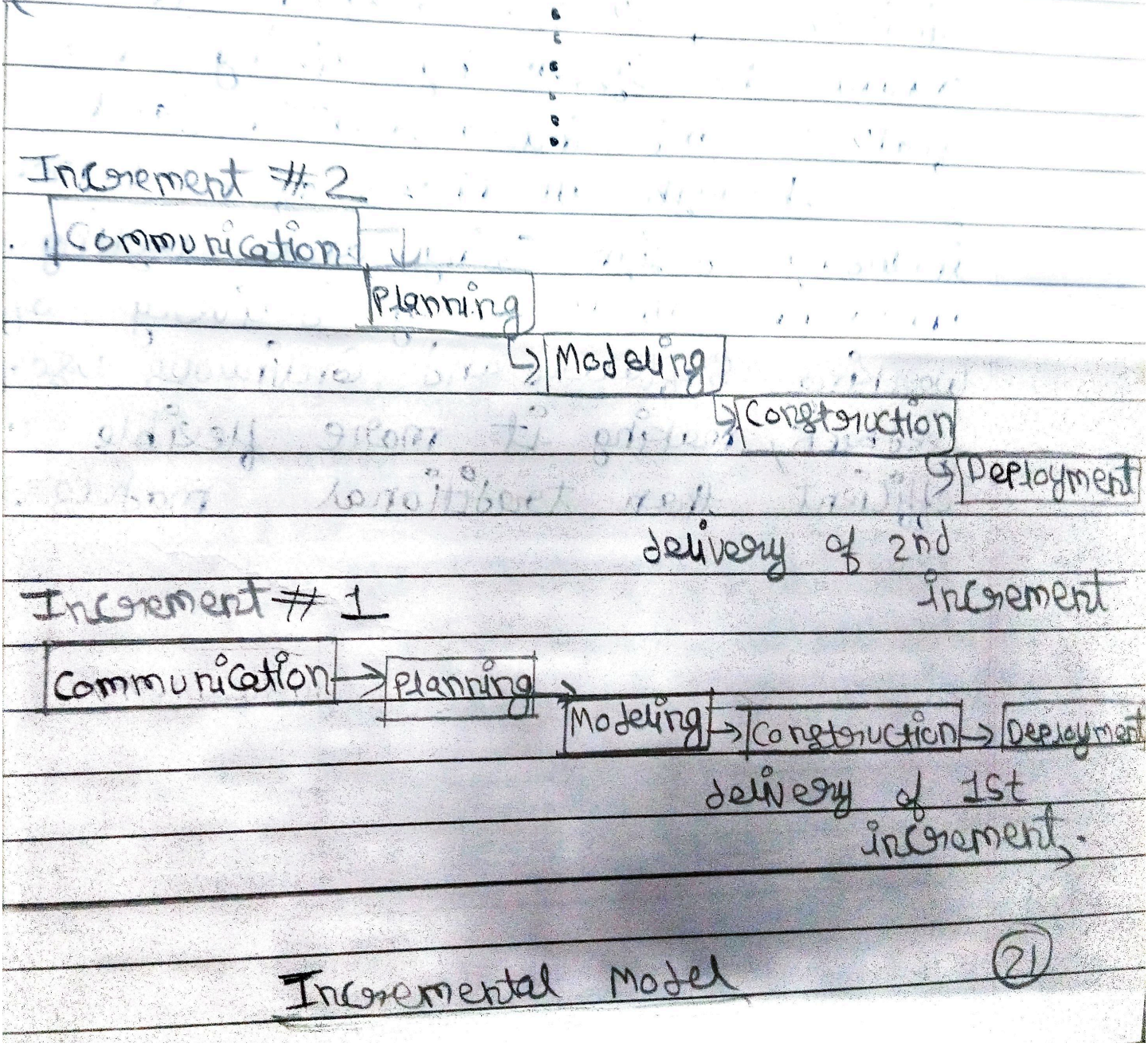


Diagram:→



Incremental Model

- Brief Descriptions:->

-> The increment model is a software development approach in which the system is developed in small parts called increments. Instead of delivering the entire software at once, the system is built and delivered in multiple stages. Each increment adds new functionality to the previous version until the complete system is developed.

-> In this model, the first increment delivers a basic version of the system with limited features. Subsequent increments enhance the system by adding more features and improvements. Each increment goes through all phases of development, including design, coding, and testing. This approach allows early delivery of working software and continuous user feedback, making it more flexible and efficient than traditional models.

- Advantages:-)

① Software is delivered in parts, allowing early use of the system.

② Easier to test and debug smaller modules compared to a full system.

③ Flexible and allow changes in later increments.

④ Reduces risk by identifying issues early in development.

- Disadvantages:-)

① Requires proper planning and design of increments

② Integration of all increments can be complex.

③ Not suitable if the system cannot be divided into smaller parts.

④ May leads to design issues if increments are not properly managed.

- When Best suited:-) This model is best suited for projects where requirements are well understood but can be implemented in parts.

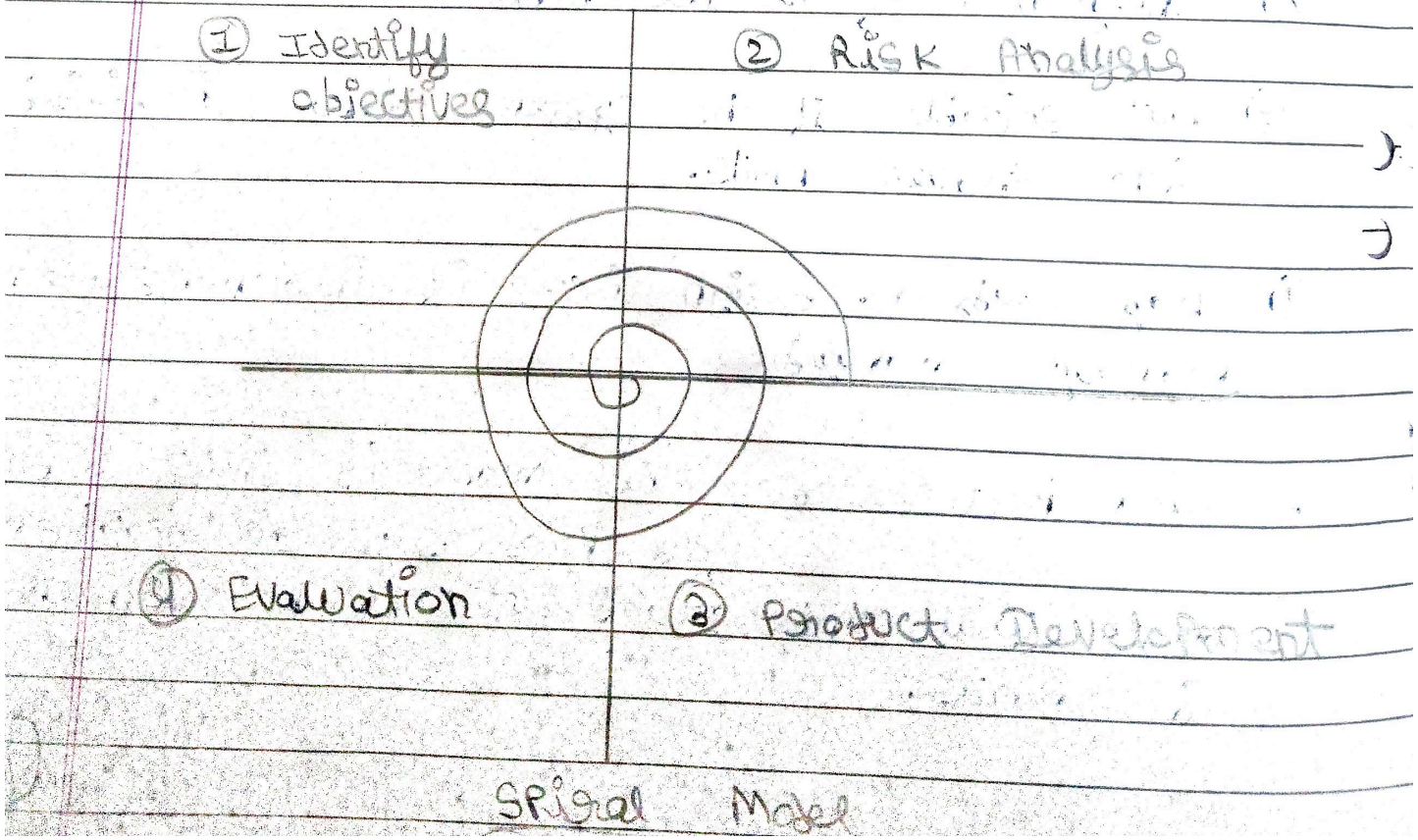
DATE
PAGE 20

⑤ Spiral Model:→

- **Year:→** The Spiral Model was introduced in the year 1986. It was developed to overcome the limitations of earlier models by combining design and Prototyping with a strong focus on risk analysis.

- **Proposed By:→** The Spiral Model was proposed by Barry Boehm, a well-known Software engineer, who introduced the concept of incorporating risk analysis into the software development process.

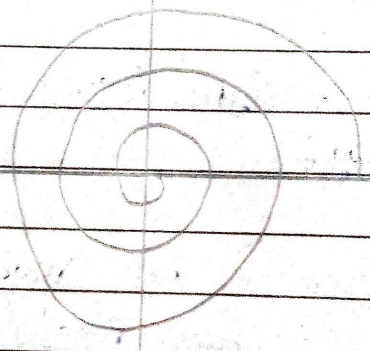
- **Diagram:→**



Spiral Model

- **Proposed By:-** The Spiral Model was proposed by Barry Boehm, a well-known Software engineer, who introduced the concept of incorporating risk analysis into the software development process.

- **Diagram:-**



Spiral Model

- **Brief Description:→**

→ The Spiral Model is an advanced Software development model that Combines elements of both iterative development and the waterfall Model, with a strong emphasis on risk analysis. In this model, development is Carried out in a spiral form, where each loop of the spiral represents a Phase of the Software Process.

→ Each Cycle of the spiral Consists of four main activities: Planning, risk analysis, engineering (development and testing), and evaluation. Before moving to the next Phase, risks are identified and analyzed Carefully. This makes the Spiral Model highly Suitable for Complex and high-risk projects. It also allows Continuous user feedback and improvements in each iteration. Although it provides better Control over risks, the model Can be expensive and requires expert knowledge to manage effectively.

- **Advantages:→**

1) Strong focus on risk analysis helps in identifying and reducing potential problems early.

flexible model that allows changes and improvements in each iteration.

Suitable for large, complex, and high-risk projects.

Continuous user feedback improves the quality of the final product.

Disadvantages:-)

Complex and difficult to understand compared to simpler models.

Requires skilled professionals for risk analysis and management.

High cost due to repeated iterations and risk evaluation.

Not suitable for small or low-budget proj

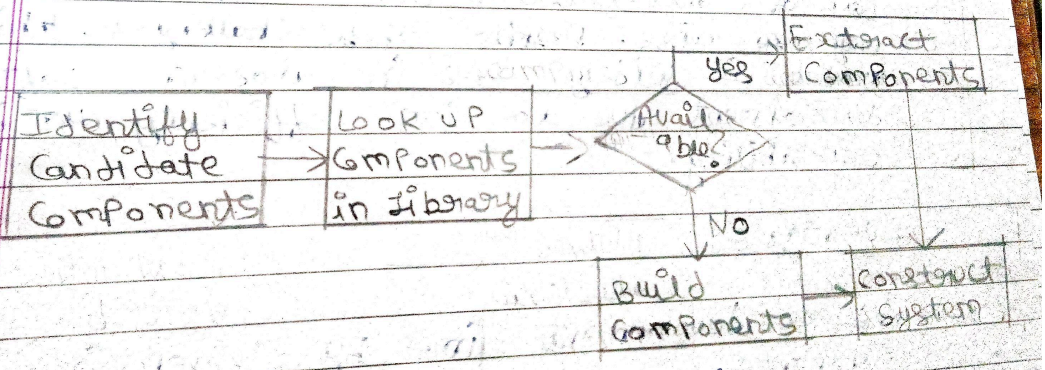
When best suited:-)

The Spiral Model is best suited for lar Complex, and high-risk projects where requirements may change frequently. It is ideal for systems that require continu evaluation and risk management throug the development process.

components.

Used By:→ This model does not have a single specific inventor. It evolved from the concept of Component-based Software engineering (CBSE), where systems are developed by assembling reusable software components.

- Diagrams:→



- Brief Description:→

(2) → The Component Assembly Model is a software development approach in which the system is built by integrating pre-existing and reusable software components. Instead of developing everything from scratch, developers identify suitable components and assemble them to create the final system.

→ This model focuses on reducing development time and cost by reusing already tested and reliable components. It improves productivity and software quality since components are often well-tested before integration. However, finding suitable components and integrating them properly can be challenging. Compatibility issues and lack of control over third-party components may also arise. Despite these challenges, this model is widely used in modern software development due to its efficiency and reusability.

- Advantages:→

(1) Reduces development time by reusing existing components.

② Improves software reliability since components are already tested.

③ Cost-effective compared to building everything from scratch.

④ Encourages modular and maintainable system design.

- Disadvantages :-)

① Difficult to find suitable components for specific requirements.

② Integration of different components can be complex.

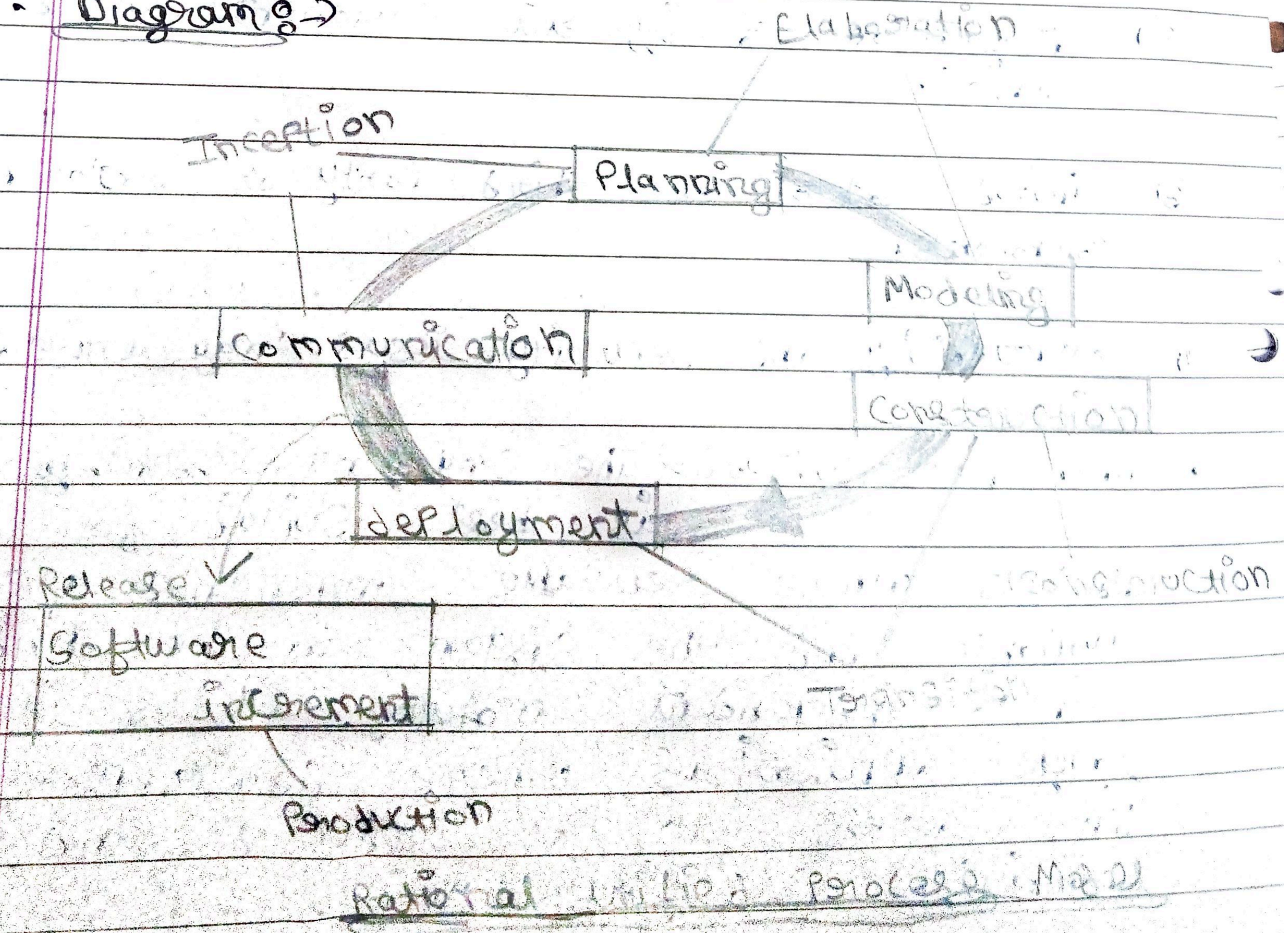
③ Limited control over third-party or external components.

④ Compatability and security issues may arise.

- When Best Suited :-) The component assembly model is best suited for projects where reusable components are available and the system can be divided into independent modules. It is ideal for large applications where development time needs to be reduced and reliability is important.

(7) RUP Model (Rational unified Process):→

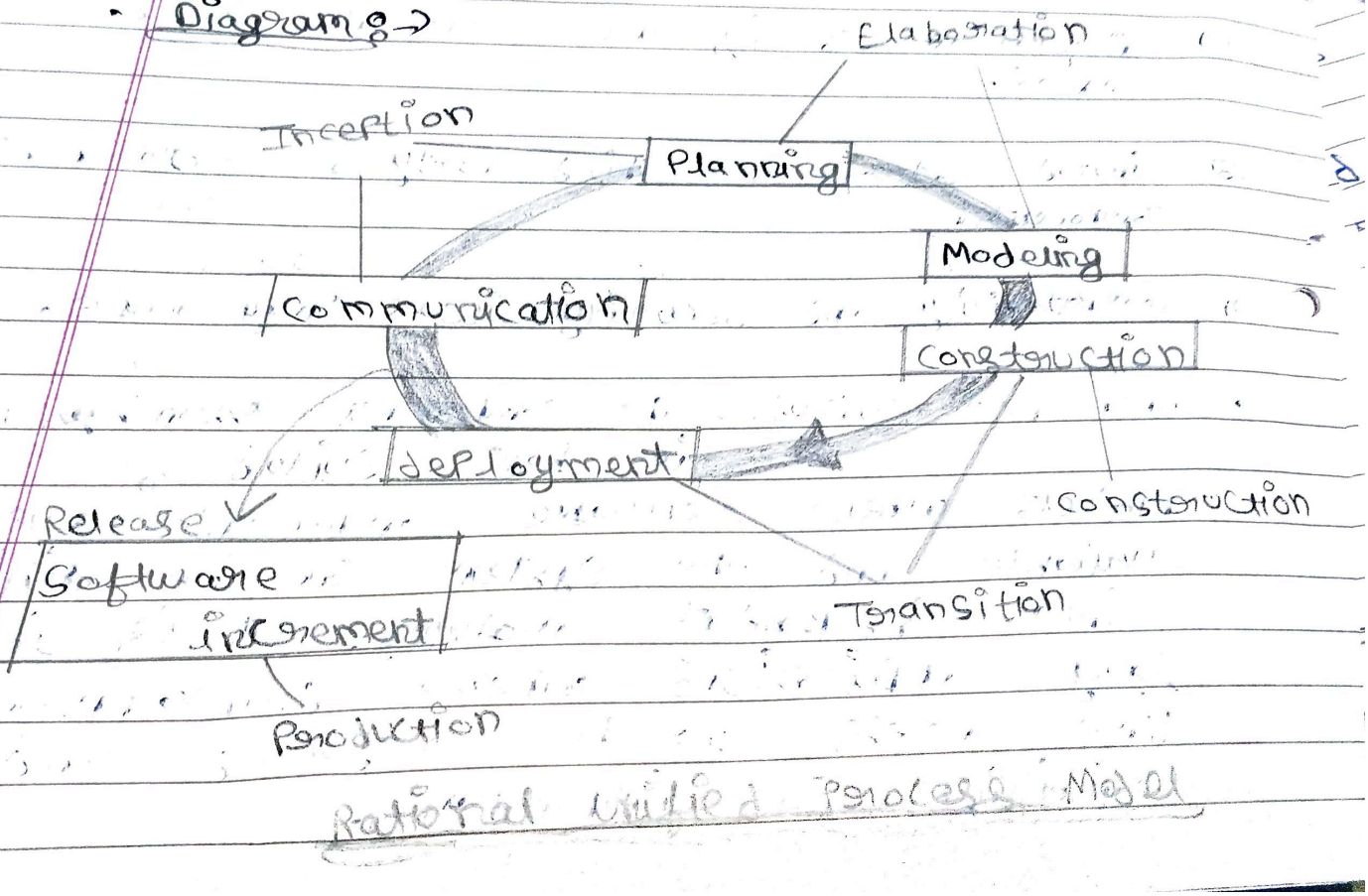
- **Year:→** The Rational unified (RUP) was developed in the 1990s and officially introduced around 1998 by Rational Software. It became widely used as an advanced and structured software development framework.
- **Proposed By:→** RUP was developed by Rational Software Corporation, which is now a part of IBM. It was designed as a comprehensive and disciplined approach to Software development.
- **Diagram:→**



Rational Unified Process (RUP)

Software development.

- Diagram →



Rational unified process Model

DATE / /
PAGE 27

- Brief Description:-

→ The Rational unified process (RUP) is an iterative and incremental software development process that emphasizes disciplined project management and high-quality software production. Unlike linear models, RUP divides the development process into multiple iterations, where each iteration improves and refines the system.

→ The RUP model consists of four main phases: Inception, where the project scope and feasibility are define; Elaboration, where system architecture and design are developed; Construction, where the actual coding and development take place; and Transition, where the software is deployed and delivered to users. Each phase can have multiple iterations, allowing continuous improvement and feedback.

- Advantages:-

(1) Provides a structured and disciplined approach to software development.

(2) Supports iterative development, allowing continuous improvement.

③ Focuses on quality through regular testing and feedback.

④ Suitable for large and complex projects with detailed planning.

- Disadvantages:-

① Complex and difficult to understand for beginners.

② Requires trained professionals and proper management.

③ Can be time-consuming and expensive to implement.

④ Not suitable for small or simple projects.

- When best suited:-

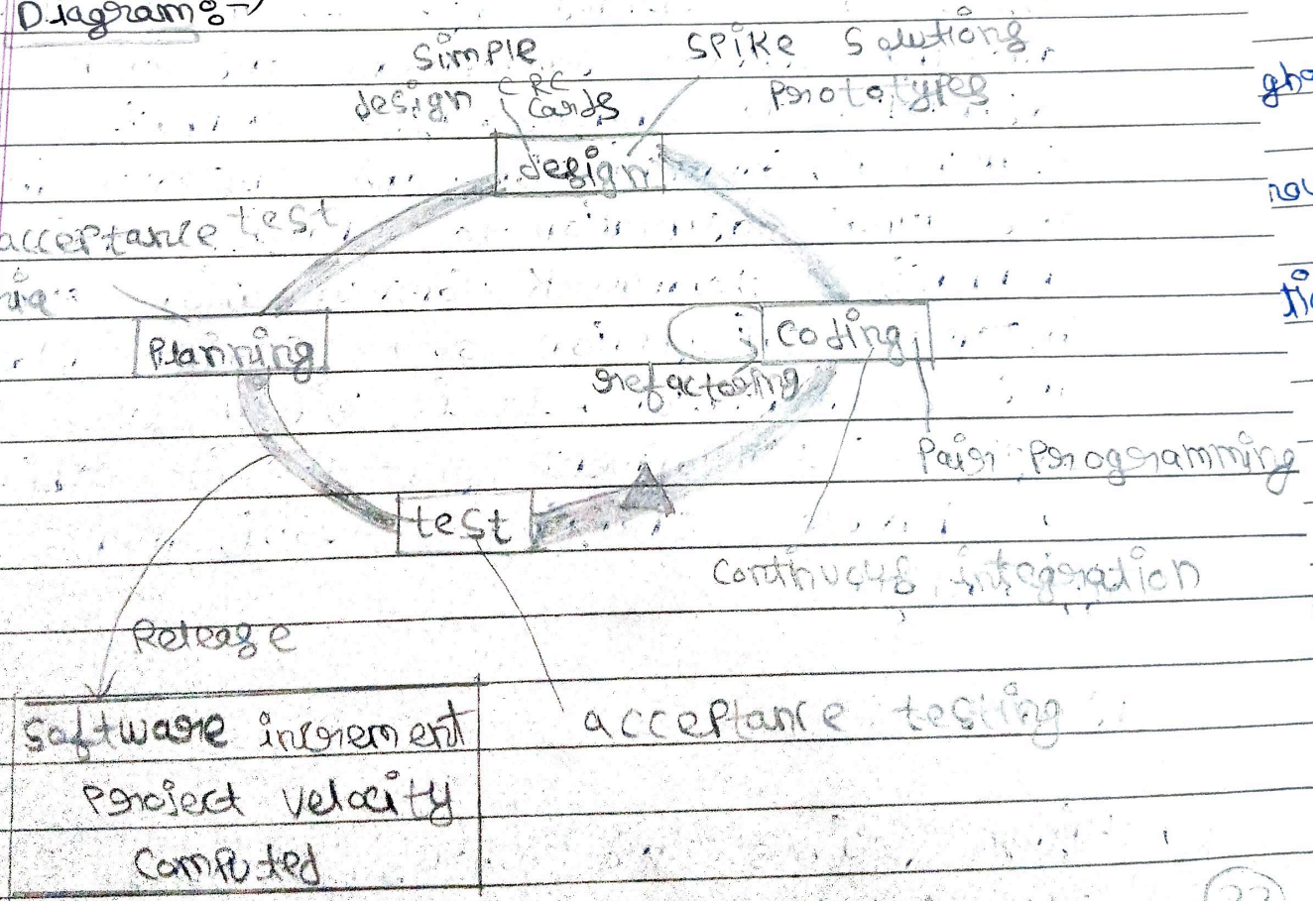
→ The RUP Model is best suited for large-scale and complex projects that require a well-defined structure, continuous feedback and high-quality standards. It is commonly used in enterprise-level software development.

(8) Agile Process Model:→

- **Years:→** The Agile Process Model was formally introduced in 2001 with the publication of the Agile Manifesto, which focused on improving flexibility and collaboration in Software development.

- **Proposed By:→** The Agile Model was proposed by a group of IT Software developers, including Kent Beck, Martin fowler, and others, who aimed to create a more flexible and adaptive development approach.

- **Diagram:→**



Agile Process Model

- Brief Description:→

→ The Agile Process Model is a modern and flexible Software development approach that focuses on iterative development, continuous feedback, and close collaboration between developers and customers. Unlike traditional models, Agile does not follow a strict sequence of phases. Instead, the development process is divided into small cycles called iterations or sprints.

→ In each sprint, a small part of the software is developed, tested, and delivered. Feedback from users is collected after every iteration, allowing continuous development and quick adaptation to changes. Agile emphasizes teamwork, communication, and customer improvement and quick adaptation to changes. Agile emphasizes teamwork, communication, and customer satisfaction over strict planning and documentation. It is widely used in software development due to its ability to handle changing requirements efficiently.

- Advantages:→

① Highly flexible and allows changes at any stage of development.

② Continuous feedback improves the quality of the software.

③ Faster delivery of working software in small parts.

④ Strong collaboration between team members and clients.

- Disadvantages:→

① Less emphasis on documentation can create issues later.

② Requires active involvement of users throughout development.

③ Difficult to manage large projects without proper planning.

④ Success depends heavily on team coordination and experience.

- When Best Suited:→

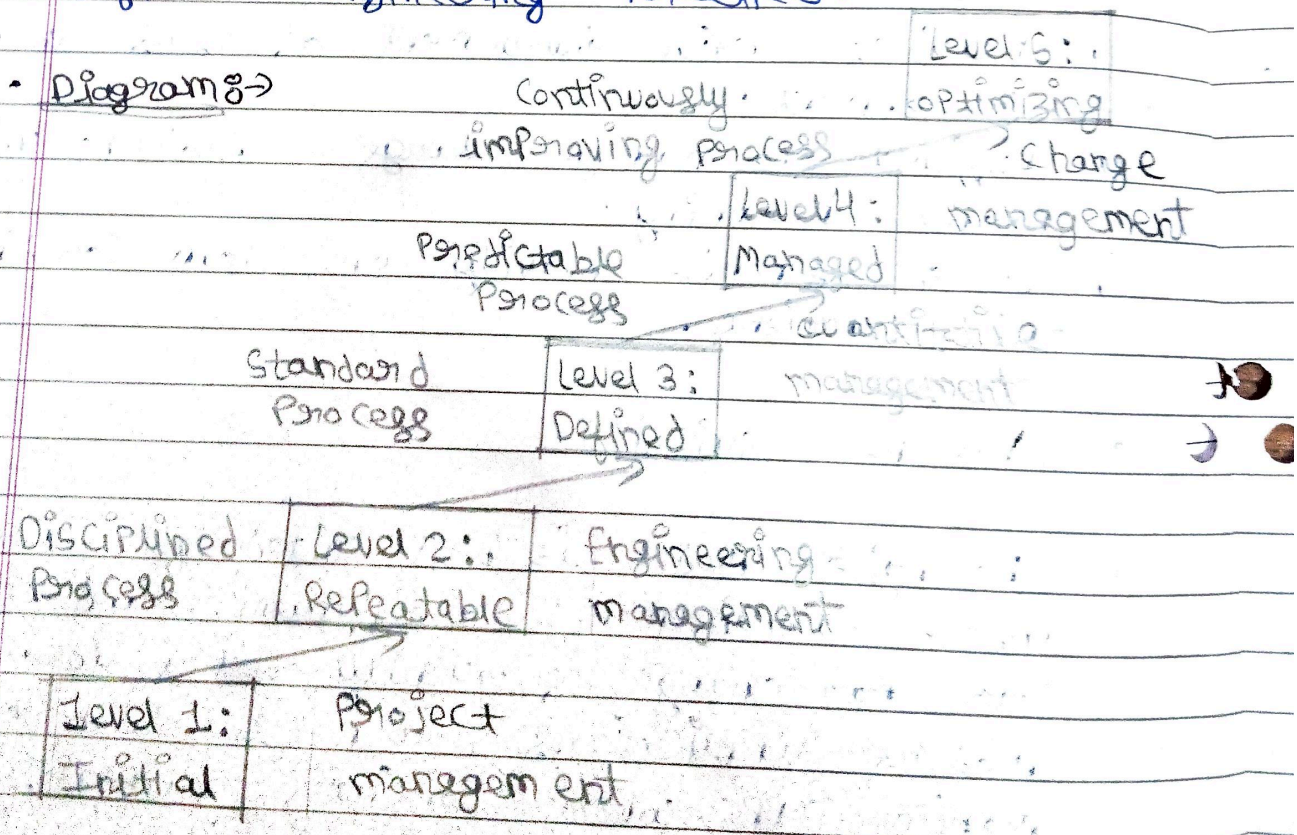
→ The Agile Model is best suited for projects where requirements are dynamic and expected to ~~than~~ change frequently. It is ideal for modern applications such as web and mobile apps, where continous updates and improvements are required.

(9) CMM (Capability Maturity Model) :→

- **Year :→** The Capability Maturity Model (CMM) was developed in the late 1980s and officially introduced around 1991. It was created to improve software development processes and ensure quality and consistency in organizations.

- **Proposed By :→** CMM was developed by the Software engineering institute, which is a research center focused on improving software engineering practices.

- **Diagram :→**



Capability Maturity Model

- **Brief Description:→**

→ The Capability Maturity Model (CMM) is a framework used to assess and improve the Software development processes of an organization. Unlike other SDLC models, CMM does not describe how to develop Software step-by-step. Instead, it focuses on evaluating the maturity level of an organization's process and guiding improvements.

→ CMM consists of five levels of maturity. At level 1 (initial), processes are unpredictable and unstructured. At level 2 (Repeatable), basic project management practices are established. At level 3 (Defined), processes are standardized across the organization. At level 4 (Managed), processes are measured and controlled. Finally, at level 5 (optimizing), continuous process improvement is achieved. This model helps organizations deliver high-quality Software consistently by improving their internal processes.

- **Advantages:→**

① Helps in improving Software quality and process efficiency.

② Provides a clear framework for process improvement.

③ Increases consistency and reliability in software development.

④ useful for large organizations and enterprise-level projects.

→ • Disadvantages:→

① Implementation can be time-consuming and costly.

② Requires strict documentation and process discipline.

③ Not suitable for small organizations or projects.

④ Focuses more on process than actual development speed.

→ • When best suited:→

→ CMM is best suited for large organizations that want to improve their software development processes and achieve high levels of quality and consistency. It is commonly used in enterprise environments and government projects.

Experiment III

PROBLEM STATEMENT: IDENTIFYING THE REQUIREMENTS

1. Problem Statement

Jharkhand is a state rich in natural beauty, tribal heritage, and cultural diversity. It includes famous tourist destinations such as Netarhat, Patratu Valley, Betla National Park, Hundru Falls, and Deoghar. Despite having such vast tourism potential, the tourism sector in Jharkhand remains underdeveloped due to several challenges.

The primary issues include lack of digital infrastructure, poor promotional strategies, low awareness among tourists, and unorganized travel and hospitality services. Tourists often struggle to find accurate and reliable information about destinations, accommodations, transportation, cultural events, and local guides.

Additionally, local artisans, tribal communities, and small-scale service providers are not effectively connected to tourists. As a result, they are unable to benefit from the tourism economy and remain excluded from digital platforms.

Therefore, there is a strong need to develop a centralized smart digital system that improves accessibility, enhances tourist experience, and promotes inclusive and sustainable tourism development in Jharkhand.

2. Solution Proposed

The proposed solution is to design and develop an AI-powered Smart Digital Tourism Platform for Jharkhand. This platform will be available as both a mobile application and a web-based system.

The platform will provide tourists with complete, reliable, and real-time information about destinations, travel services, accommodations, and cultural activities.

It will include advanced features such as:

- AI-based personalized itinerary planning based on user preferences
- Chatbot assistance for guiding users
- Interactive maps and navigation support
- Real-time transport tracking
- Blockchain-based secure transactions and service verification
- Online marketplace for local handicrafts and services
- Analytics dashboard for government authorities

This system will not only improve the tourist experience but also empower local communities and promote sustainable tourism growth.

3. Users

The system will support multiple types of users:

3.1 Tourists

- Explore destinations
- Plan trips
- Book hotels and transport
- Get personalized recommendations

3.2 Service Providers

- Hotel owners
- Transport operators
- Tour guides

They can:

- Register services
- Manage bookings
- Promote offerings

3.3 Local Artisans & Communities

- Sell handicrafts
- Promote cultural events
- Participate in tourism economy

3.4 Government Authorities / Admin

- Monitor tourism activities
- Analyze user data
- Generate reports
- Improve infrastructure planning

4. Functionalities

4.1 Identification of functional requirements.

The system should provide the following functionalities:

1. User registration and login
2. AI-based itinerary planning
3. Search and explore tourist destinations
4. Booking system for hotels and transportation
5. Real-time navigation and tracking
6. Chatbot assistance for users
7. Vendor/service provider registration

8. Online marketplace for local products
9. Feedback and rating system
10. Admin dashboard for monitoring and control

Table 01: Identifier and priority for software requirements

#	Requirement	Priority
R1	User registration and login	High
R2	AI-based itinerary planning	High
R3	Destination search and exploration	High
R4	Booking system	High
R5	Real-time navigation	Medium
R6	Vendor registration	Medium
R7	Online marketplace	Medium
R8	Chatbot Assistance	Medium
R9	Feedback System	Medium
R10	Admin Dashboard	High

4.2 . Identification of non-functional requirements.

The system should satisfy the following non-functional requirements:

4.2.1. Performance

The system should respond quickly and handle multiple users simultaneously without delay.

4.2.2. Scalability

The system should be able to handle increasing users and data as tourism grows.

4.2.3. Security

User data and transactions must be secure using encryption and authentication.

4.2.4. Privacy

User information should be protected and not shared without permission.

4.2.5. Usability

The system should be user-friendly and easy to navigate for all users.

4.2.6. Availability

The platform should be available 24/7 without downtime.

4.2.7. Reliability

The system should provide accurate results and work without failures.

4.2.8. Maintainability

The system should be easy to update and improve.

4.2.9. Accessibility

The platform should support different users including differently-abled users.

4.2.10. Multilingual Support

The system should support multiple languages for better usability.

Experiment IV

MODELING UML USE CASE DIAGRAMS SPECIFICATIONS

Introduction

Jharkhand is blessed with natural beauty, tribal culture, historical landmarks, and eco-tourism hotspots like Netarhat, Patratu, Betla National Park, Hundru Falls, and Deoghar. However, the state's tourism industry is underdeveloped due to a lack of digital infrastructure, limited promotional outreach, and unorganized travel services. Tourists face challenges in accessing reliable details about destinations, transport, and certified guides.

To solve these issues, a centralized AI-powered digital tourism platform is proposed. The platform will offer personalized itinerary planning, multilingual chatbot assistance, real-time transport and location tracking, and an integrated marketplace for tribal handicrafts and homestays. It will also leverage blockchain for secure transactions and guide verification.

The final deliverable will be a web and mobile platform that provides a unified, secure, and user-friendly ecosystem to promote inclusive, organized, and sustainable tourism in Jharkhand.

Problem Statement

Jharkhand tourism faces challenges due to limited digital infrastructure, lack of proper destination awareness, and unorganized tourism services. Tourists struggle to find verified information about destinations, guides, accommodations, transportation, cultural events, and local handicrafts through a single reliable platform.

A smart digital platform is proposed to connect tourists, service providers, and tourism officials by offering AI recommendations, booking services, secure transactions, service verification, marketplace support, and tourism analytics. The objective of this experiment is to model the system using UML Use Case Diagrams.

Table 01: List of Actors

Actor	Description
Tourist	Explores destinations, plans AI itinerary, books services, purchases marketplace items, makes payments, and interacts with AI chatbot.
Local Service Provider	Registers services, lists handicrafts, homestays and events, manages bookings, and uploads verification documents.

Tourism Official (Admin)	Verifies service providers, manages certifications, monitors analytics dashboard, and generates tourism reports.
Payment Gateway	Handles secure transaction authentication and payment processing.

Table 02: List of Use Cases

#	Use Case	Description
UC1	Register / Login	Allows tourists, service providers, and officials to create accounts and securely authenticate.
UC2	Browse Destinations	Enables tourists to explore eco-tourism locations, cultural sites, and destination information.
UC3	Plan Personalized AI Itinerary	Generates customized travel plans based on tourist preferences and interests.
UC4	Chat with Multilingual Assistant	Provide AI chatbot support and multilingual assistance for tourists.
UC5	View Interactive Maps & AR/VR Preview	Displays interactive maps and immersive virtual previews of tourist destinations.
UC6	Book Tourism Services	Allows booking of guides, homestays, events, and tourism services.
UC7	Buy & Sell Local Marketplace Items	Enables listing and purchasing of tribal handicrafts, cultural products, and local offerings.
UC8	Make Secure Payment	Handles secure transactions using payment gateway and blockchain-enabled verification.
UC9	Verify Service Provider & Manage Certifications	Verifies local guides and service providers and manages digital certification records.
UC10	Monitor Analytics Dashboard	Provides tourism officials with reports, trends, feedback analysis, and tourism insights.

Table 03: Mapping of Functional Requirements to Use Cases

FR#	Functional Requirement	Use Case(s)
R1	User Registration and Authentication	UC1
R2	Destination Browsing and Information Access	UC2

R3	AI-Based Personalized Itinerary Planning	UC3
R4	Multilingual Chatbot Assistance	UC4
R5	Interactive Maps and AR/VR Preview	UC5
R6	Tourism Service Booking Management	UC6
R7	Local Marketplace Management	UC7
R8	Secure Payment and Blockchain Verification	UC8
R9	Service Provider Verification and Certification	UC9
R10	Tourism Analytics and Report Generation	UC10

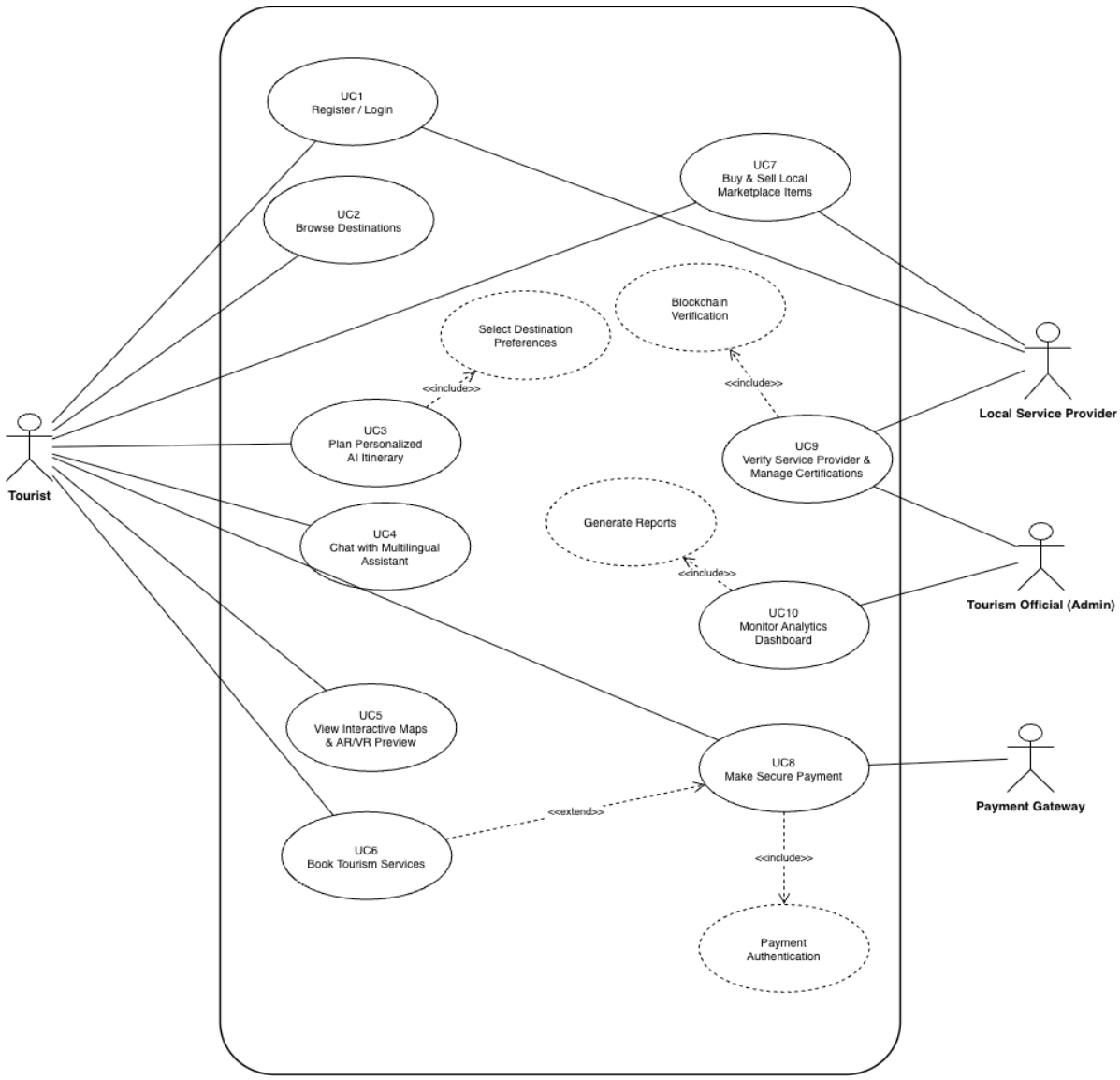


Figure 01 : USE CASE DIAGRAM

Use Case Specifications :

Table 04: UC-01: Register/Login :

Field	Description
Use Case	UC1. Register / Login
Description	Allows tourists and local service providers to create an account and log in.
Assumptions	User has a valid mobile number or email address.
Actors	Tourist, Local Service Provider
Steps	1. User enters credentials. 2. System validates input 3. System logs user in.
Variations	Invalid details or missing documents prompt error messages.
Non-functional	Secure storage and processing under 2 seconds.
Issues	Delay in receiving OTP or email verification links.

Table 05 : UC-02: Browse Destinations :

Field	Description
Use Case	UC2. Browse Destinations
Description	Displays information, photos, and features of tourism hotspots in Jharkhand.
Assumptions	Destination database is populated and online.
Actors	Tourist
Steps	1. Tourist searches/filters destinations. 2. System displays search results. 3. Tourist views destination detail page.
Variations	No results found displays default popular destinations.
Non-functional	Destination search results display under 1 second.
Issues	Incomplete database details for remote tourist sites.

Table 06: UC-03: Plan Personalized AI Itinerary :

Field	Description
Use Case	UC3. Plan Personalized AI Itinerary
Description	Generates custom itineraries based on user preferences and travel details.
Assumptions	AI itinerary engine is functional.
Actors	Tourist
Steps	1. Tourist enters travel preferences. 2. AI engine processes inputs. 3. Platform renders custom itinerary.
Variations	AI suggests default popular itineraries if input is too vague.
Non-functional	Itinerary generation time should be less than 3 seconds.
Issues	Lack of historical user data or recommendations for off-beat eco-tourism spots.

Table 07 : UC-04: Chat with Multilingual Assistant :

Field	Description
Use Case	UC4. Chat with Multilingual Assistant
Description	Assists tourists with real-time queries in English, Hindi, and local tribal languages.
Assumptions	Chat bot engine is online and connected.
Actors	Tourist
Steps	1. Tourist asks query in chosen language. 2. System parses language and query. 3. Chatbot returns translated answer
Variations	Falls back to English if regional dialect is not supported.
Non-functional	Chatbot response latency under 1.5 seconds.
Issues	Accurate translation of tribal dialects and offline availability.

Table 08 : UC-05: View Interactive Maps & AR/VR Preview:

Field	Description
Use Case	UC5. View Interactive Maps & AR/VR Preview
Description	Renders interactive geographical maps and immersive 3D/virtual reality previews of destinations.
Assumptions	User device has gyroscope/accelerometer and supports AR/VR.
Actors	Tourist
Steps	1. Tourist selects destination. 2. Clicks AR/VR preview. 3. System loads and displays interactive 3D tour.
Variations	Fallback to standard 2D image gallery if device lacks AR support.
Non-functional	AR/VR rendering should maintain 60 FPS for smooth user experience.
Issues	High bandwidth consumption and device heating issues.

Table 09 : UC-06: Book Tourism Services :

Field	Description
Use Case	UC6. Book Tourism Services
Description	Allows booking of verified local guides, tours, and homestay accommodations.
Assumptions	Service listing is available and active in the system.
Actors	Tourist
Steps	1. Tourist selects service and dates. 2. System checks availability. 3. Redirects to secure blockchain payment.
Variations	Double booking prevention; booking cancellation.
Non-functional	Booking transactions must be consistent and highly available.
Issues	Communication delay with remote local service providers.

Table 10: UC-07: Buy and Sell Local Marketplace Items:

Field	Description
Use Case	UC7. Buy & Sell Local Marketplace Items
Description	Enables local artisans and providers to list and sell tribal handicrafts, event tickets, and homestays.
Assumptions	Provider is registered; marketplace catalog is active.
Actors	Tourist, Local Service Provider
Steps	1. Provider lists items and pricing. 2. Tourist browses and adds to cart 3. Tourist browses and adds to cart
Variations	Out-of-stock items show warning messages.
Non-functional	Product catalog search should return results in under 1 second.
Issues	Delivery and logistics coordination in remote rural areas of Jharkhand.

Table 11 : UC-08: Make Secure Blockchain Payment:

Field	Description
Use Case	UC8. Make Secure Payment
Description	Processes secure monetary transactions and marketplace escrow payouts via blockchain smart contracts.
Assumptions	Payment details are linked; payment gateway is active.
Actors	Tourist, Payment Gateway
Steps	1. User confirms transaction. 2. System deploys payment transaction. 3. System records receipt on blockchain.
Variations	No results found displays default popular destinations.
Non-functional	End-to-end encryption of transaction details using secure protocols.
Issues	Blockchain network congestion or gas fees.

Table 12 : UC-09: Verify Guide & Manage Certifications:

Field	Description
Use Case	UC9. Verify Service Provider & Manage Certifications
Description	Verifies local tour guides and registers their certificates on the blockchain ledger.
Assumptions	Official has authority to review; guide has submitted credentials.
Actors	Local Service Provider, Tourism Official (Admin)
Steps	1. Guide uploads government ID and certifications. 2. Tourism Official reviews and approves. 3. System writes verified certification to blockchain.
Variations	Rejection of certification if documents are invalid or expired.
Non-functional	Immutability of guide records on the blockchain network.
Issues	Delay in blockchain block validation and transaction confirmation.

Table 13 : UC-10: Monitor Analytics Dashboard:

Field	Description
Use Case	UC10. Monitor Analytics Dashboard
Description	Renders charts, analytics, and reports to monitor tourist inflow and local marketplace sales.
Assumptions	Official is logged into the admin dashboard.
Actors	Tourism Official (Admin)
Steps	1. Official navigates to dashboard 2. System retrieves real-time data. 3. System renders analytics reports.
Variations	Custom date filters available for specific hotspots.
Non-functional	Dashboard load time under 2 seconds.
Issues	Data lag from remote regions.

Important UML Relationships:

«include» Relationship

- Plan Personalized AI Itinerary «include» Select Destination Preferences
- Make Secure Payment «include» Payment Authentication

«extend» Relationship

- Book Tourism Services «extend» Make Secure Payment
- Monitor Analytics Dashboard «include» Generate Reports

Generalization

- Verify Service Provider & Manage Certifications «include» Blockchain Verification

Experiment V
Synopsis

1. Introduction of the Project

Jharkhand is one of India's richest states in terms of natural beauty, tribal culture, and eco-tourism potential, yet it remains underexplored due to the lack of a centralized digital platform. Tourists face difficulties in discovering destinations, planning itineraries, booking local services, and making secure payments. Local service providers and artisans lack digital visibility, while tourism officials have no unified analytics system. To address these gaps, an AI-powered Smart Digital Platform for Eco & Cultural Tourism in Jharkhand is proposed. This platform integrates personalized itinerary planning, a multilingual chatbot, AR/VR destination previews, a local marketplace, blockchain-based secure payments, guide verification, and an analytics dashboard — all on a single web and mobile application.

2. Objective

The main objective of this project is to develop an AI-powered web and mobile platform that promotes eco and cultural tourism in Jharkhand by providing tourists with personalized travel planning, multilingual support, AR/VR destination previews, seamless service booking, and secure blockchain payments, while also empowering local service providers with digital marketplace access and equipping tourism officials with a real-time analytics dashboard.

3. Scope

The platform targets domestic and international tourists visiting Jharkhand, local service providers including guides, homestay owners, and artisans, as well as tourism officials. Key features include AI itinerary planning, interactive maps, AR/VR previews, a tribal handicraft marketplace, blockchain payment verification, guide certification management, multilingual chatbot, and an official analytics dashboard. The system covers the entire tourism lifecycle — from discovery and planning to booking, payment, and post-visit feedback.

4. Study of Existing System

Several travel and tourism applications already exist and are widely used by tourists for planning trips, booking accommodation, and navigation. However, none of these platforms are built specifically for Jharkhand's eco and cultural tourism needs, and none combine destination discovery, AI-based itinerary planning, local service provider onboarding, a tribal handicrafts marketplace, and government analytics into a single system. Five existing applications relevant to this domain were studied to identify gaps that the proposed platform addresses.

Existing System/Application 1: MakeMyTrip

Problems Addressed

Hotel booking, flight and transport booking, and holiday package planning for domestic and international travel.

Advantages

- Wide network of hotels, flights, and travel packages.
- Established online payment and booking system.
- Customer reviews and ratings for accommodations.

Disadvantages

- Not focused on regional eco or cultural tourism.
- No support for local tribal artisans or handicrafts.
- No AI-based personalised itinerary planning for a specific state.

Gaps Identified

MakeMyTrip is a general-purpose booking platform and does not provide destination-specific guidance, cultural event information, or a marketplace for local artisans in a region such as Jharkhand.

Reference

https://www.makemytrip.com

Existing System/Application 2: TripAdvisor

Problems Addressed

Destination reviews, ratings, and travel recommendations from other travellers.

Advantages

- Large database of user reviews and ratings.
- Useful for comparing hotels, restaurants, and attractions.
- Helpful travel forums and community discussions.

Disadvantages

- Reviews can be outdated or inconsistent for lesser-known destinations.
- No real-time booking, transport tracking, or payment integration.
- No dedicated support for local service providers or artisans.

Gaps Identified

TripAdvisor is review-driven rather than transactional, so it cannot generate personalised itineraries, process bookings, or connect tourists directly with verified local service providers.

Reference

https://www.tripadvisor.in

Existing System/Application 3: Google Maps

Problems Addressed

Navigation, route guidance, real-time traffic information, and location sharing.

Advantages

- Accurate GPS navigation and live traffic updates.
- Extensive coverage of roads and points of interest.
- Easy location sharing between users.

Disadvantages

- No tourism-specific itinerary planning or booking features.
- No marketplace, service provider management, or payment system.
- No analytics dashboard for tourism authorities.

Gaps Identified

Google Maps provides navigation but cannot recommend a complete trip plan, manage bookings, or give tourism departments visibility into tourist movement and destination popularity.

Reference

https://maps.google.com

Existing System/Application 4: Airbnb

Problems Addressed

Booking of homestays, guesthouses, and local accommodation experiences.

Advantages

- Wide variety of accommodation options and local experiences.
- Direct communication between hosts and guests.
- Secure online payment for bookings.

Disadvantages

- Limited presence of listings in smaller Jharkhand destinations.
- No integration with local transport, itinerary planning, or handicrafts.
- No government-facing analytics or verification of service providers.

Gaps Identified

Airbnb solves only the accommodation part of the tourism journey and does not address itinerary planning, transport, or local artisan support, which are central to this project.

Reference

https://www.airbnb.com

Existing System/Application 5: Incredible India

Problems Addressed

National-level tourism promotion and general information about destinations across India.

Advantages

- Official government platform with credible tourism content.
- Wide coverage of destinations across all Indian states.
- Useful for general awareness and promotional campaigns.

Disadvantages

- Content is broad and not personalised for individual tourists.
- No booking, payment, chatbot, or itinerary planning features.
- No dedicated marketplace or service provider management for a state.

Gaps Identified

Incredible India promotes tourism at a national level but does not offer a transactional, state-specific platform with AI planning, bookings, and local business integration for Jharkhand.

Reference

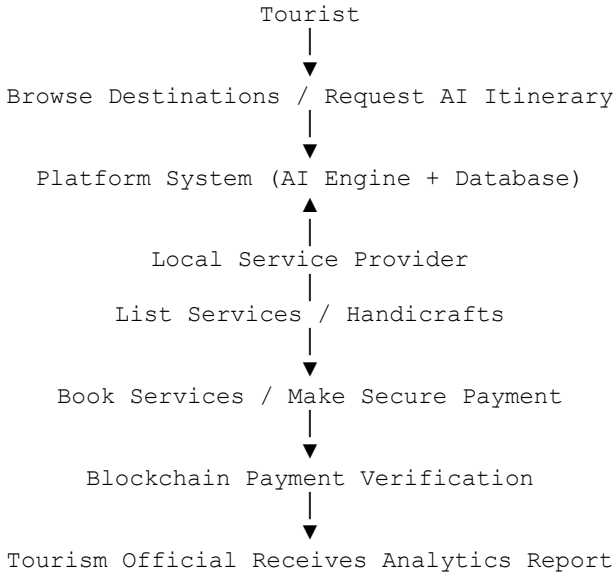
https://www.incredibleindia.gov.in

5. Project Description

The AI-Powered Smart Digital Platform for Eco & Cultural Tourism in Jharkhand is a web and mobile application designed to transform how tourists discover and experience Jharkhand. Tourists can register, browse eco-tourism and cultural destinations, and receive AI-generated personalized itineraries based on their preferences, budget, and travel duration. An AR/VR preview feature allows virtual exploration of destinations before visiting.

Local service providers can list their services — homestays, handicraft products, guide services, and cultural events — on the integrated marketplace. Tourists can book services, make secure payments through blockchain-enabled gateways, and interact with a multilingual AI chatbot for real-time assistance in English, Hindi, and local tribal languages. Tourism officials can monitor platform analytics, verify guide certifications, and generate reports through a dedicated admin dashboard.

Information Flow



6. Methodology / Planning of Project Work

The project will be developed using an Agile methodology so that features can be built and tested incrementally over an eight-week schedule. The first week focuses on requirement gathering and problem analysis based on the identified use cases and actors. The second week covers system design, including the UML use case diagram and ER diagram, followed by UI/UX design for the mobile application and web dashboard in the third week. Backend development, including the database and core APIs, begins in the fourth week alongside the destination and itinerary modules. AI-based itinerary recommendation and the chatbot assistant are developed in the fifth week, while booking, payment, and the service provider and admin dashboards are completed in the sixth week. The seventh week is dedicated to unit, integration, and system testing to ensure the platform functions reliably, and the final week covers deployment, documentation, and a demonstration of the completed system. This structured plan ensures that core functionality is available early and refined progressively through each phase.

Table 01: Project Development Schedule

Week	Activity	Deliverable
Week 1	Requirement gathering and problem analysis	Requirements finalised
Week 2	System design, UML use case and ER diagrams	Design completed
Week 3	UI/UX design for mobile app and web dashboard	Interface designs completed

Week 4	Backend and database development, destination module	Core backend completed
Week 5	AI itinerary engine and chatbot assistant	Smart planning module completed
Week 6	Booking, payment, service provider and admin dashboard	Functional modules integrated
Week 7	Unit testing, integration testing, and bug fixing	Stable system ready
Week 8	Deployment, documentation, and final demonstration	Project completed

Table 02 : Project Timeline (Gantt Chart)

Task	W1	W2	W3	W4	W5	W6	W7	W8
Requirement Analysis	●	●						
System Design		●	●					
UI/UX Design			●	●				
Backend Development				●	●			
AI & Chatbot Integration					●	●		
Booking & Dashboards						●		
Testing							●	
Deployment & Documentation								●

7. Features

User Interface Features

- Simple and responsive mobile application for tourists.
- Interactive web dashboard for service providers and administrators.
- Multilingual interface supporting English and regional languages.
- Map-based visualisation for easy navigation between destinations.

Functional Features

- Tourist registration and secure login.
- AI-based personalised itinerary planning.
- AI chatbot for trip-related assistance.
- AR/VR preview of selected destinations.
- Real-time transport information and tracking.
- Online booking of services and secure digital payments.

- Marketplace for tribal handicrafts and local products.
- Service provider registration and listing management.
- Feedback and rating system for tourists.
- Admin dashboard for verifying providers, and managing destinations and events.

Security Features

- Secure authentication for tourists, service providers, and admins.
- Encrypted storage of user data and payment details.
- Role-based access control for different user types.
- Verification workflow for onboarding service providers.

Performance Features

- Fast loading of destination and itinerary data.
- Efficient handling of concurrent bookings and payments.
- Scalable backend architecture for growing user demand.
- Optimised AI recommendation response time.

Reporting & Analytics

- Analytics dashboard showing tourist activity and popular destinations.
- Booking and revenue reports for administrators.
- Feedback summary reports for service quality monitoring.
- Event and destination management reports.

User Experience Enhancements

- Push notifications for bookings, events, and offers.
- Voice-assisted chatbot support for ease of use.
- Personalised recommendations based on tourist preferences.
- Simple, accessible interface suitable for first-time users.

8. System Architecture

The proposed system follows a multi-tier architecture consisting of a presentation layer, an application layer, an AI services layer, and a data layer. Tourists interact with the platform through the mobile application, while service providers and administrators access the web dashboard. The application server handles user authentication, destination browsing, itinerary generation, booking, and notification services. The AI services layer powers the personalised itinerary engine and the chatbot assistant, processing tourist preferences to generate recommendations. The data layer stores tourist profiles, itineraries, destinations,

bookings, payments, feedback, events, and reports, corresponding to the entities identified in the ER diagram. External services such as maps, the payment gateway, and notification APIs communicate with the application server to support navigation, transactions, and real-time updates. This layered design improves scalability, maintainability, and security across the platform.

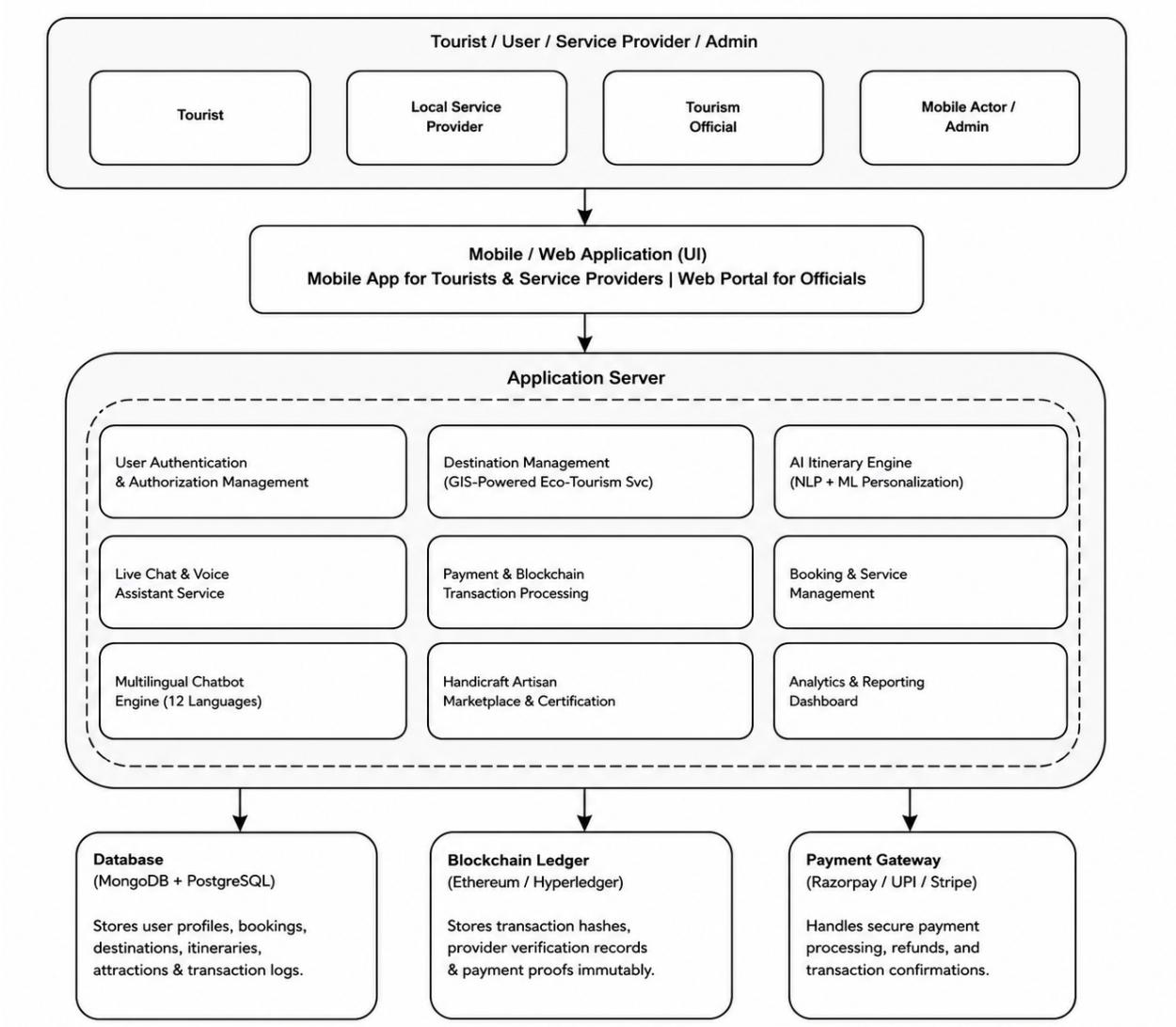


Figure 01 : System Architecture Diagram

9. User Interface

The platform is designed with a clean and intuitive interface so that tourists, local service providers, and tourism officials can easily navigate and use all features without technical expertise.

Main Screens

9.1. Splash Screen

- Platform logo and tagline
- Welcome animation and language selection screen

9.2. Login / Registration

- Mobile Number / Email
- Password and OTP Verification
- Register as Tourist or Service Provider
- Forgot Password / Social Login

9.3. Tourist Dashboard

- AI Itinerary Planner My Bookings
- Browse Destinations
- Marketplace
- Payments & Receipts
- Profile & Language Settings

9.4. Service Provider Dashboard

- Manage Listings
- Bookings Received
- Upload Certifications
- Payments Profile

9.5. Destination / Service Details

- Destination Image / AR-VR Preview
- Guide / Host Details
- Availability Calendar
- Pricing
- Tourist Reviews & Ratings
- AI Itinerary Suggestion
- Book Now

9.6. Payment Screen

- Booking Summary
- Payment Method (UPI / Card / Blockchain Wallet)
- Blockchain Payment Confirmation & Receipt

9.7. Booking Status Tracker Requested

- Confirmed
- In Progress / Active
- Completed

10. Technology Stack

Table 03 : Technology Stack

Component	Technology
Frontend	React.js / React Native
Backend	Node.js + Express.js
Database	MongoDB / PostgreSQL
Authentication	JWT + OTP Verification
Payment Gateway	Razorpay / Stripe + Blockchain Smart Contracts
Blockchain	Ethereum / Hyperledger Fabric
AI Engine	Python (TensorFlow / OpenAI API)
AR/VR	Unity 3D / A-Frame WebXR
Version Control	Git & GitHub

11. Testing Plan

The platform will undergo multiple levels of testing to ensure reliability, security, and performance. Unit testing will validate individual modules such as the AI itinerary engine, multilingual chatbot, AR/VR preview renderer, and blockchain payment gateway. Integration testing will verify seamless data flow between the tourist app, service provider portal, and admin dashboard. Security testing will cover authentication, payment encryption, and blockchain transaction integrity. Performance testing will ensure the AI itinerary generation responds within 3 seconds and the analytics dashboard loads in under 2 seconds under concurrent user load.

Table 04 : Testing Plan Table

Test Case ID	Test Scenario	Expected Result
TC01	Tourist Registration	Account created successfully
TC02	Tourist Login	Login successful with valid credentials
TC03	AI Itinerary Generation	Personalized itinerary displayed within 3 seconds
TC04	Browse Destinations	Destination list and details displayed correctly
TC05	AR/VR Preview	Immersive destination preview renders correctly
TC06	Book Tourism Service	Booking confirmed and notification sent
TC07	Blockchain Payment	Secure payment processed and receipt generated
TC08	Booking Status Tracking	Correct booking status displayed in real-time

TC09	Guide Certification Verify	Certification verified and recorded on blockchain
TC10	Analytics Dashboard Load	Dashboard loads with correct data in under 2 seconds

12. Expected Outcome (100–150 words)

The AI-Powered Smart Digital Platform for Eco & Cultural Tourism in Jharkhand is expected to significantly improve the visibility of Jharkhand's eco and cultural tourism on a national and international level. Tourists will benefit from a seamless, personalized travel experience with AI itinerary planning, AR/VR previews, and multilingual chatbot support. Local artisans and service providers will gain digital marketplace access, increasing their income and economic participation. Tourism officials will receive real-time analytics and reports, enabling data-driven policy decisions. The platform is expected to increase tourist footfall, promote tribal culture and handicrafts, and create a transparent and secure tourism ecosystem powered by AI and blockchain technology.

13. Resources and Limitations (100–150 words)

Resources Required

Hardware

- Laptop/Desktop for development
- Android/iOS Smartphone for testing
- Stable Internet Connection

Software

- React.js / React Native (Frontend)
- Node.js with Express.js (Backend)
- MongoDB / PostgreSQL (Database)
- Python (AI Itinerary Engine & Chatbot)
- Git & GitHub (Version Control)
- Ethereum / Hyperledger (Blockchain Payments)
- Razorpay / Stripe API (Payment Gateway)

Data Resources

- Jharkhand tourism destination data and images
- Local service provider listings and pricing
- Tourist preference and feedback data for AI training

Limitations

The current scope focuses on Jharkhand's eco and cultural tourism and does not cover other Indian states. AR/VR features require devices with gyroscope support. Blockchain transactions may face network congestion or latency issues. Tribal dialect support in the multilingual chatbot is limited to major languages in the initial version. Remote areas of Jharkhand with poor internet connectivity may face reduced platform functionality.

14. Conclusion

The AI-Powered Smart Digital Platform for Eco & Cultural Tourism in Jharkhand is designed to bridge the gap between Jharkhand's rich natural and cultural heritage and the modern digital traveler. By integrating artificial intelligence, blockchain technology, AR/VR experiences, and a local marketplace, the platform empowers tourists, local service providers, and tourism officials alike. It provides a unified, secure, and user-friendly ecosystem that promotes sustainable eco-tourism, preserves tribal culture, and drives economic growth in Jharkhand through responsible digital tourism.

15. References

Use a consistent citation style (APA or IEEE). Suitable references for this project:

1. Ministry of Tourism, Government of India – National Tourism Policy and Reports.
2. Jharkhand Tourism Department – Official eco and cultural tourism information.
3. Smart India Hackathon (SIH) 2025 – Problem Statement Documentation.
4. MakeMyTrip – Online travel booking platform. www.makemytrip.com
5. TripAdvisor – Travel destination reviews and guides. www.tripadvisor.com
6. Google Maps Platform – Location and mapping APIs. developers.google.com/maps
7. Ethereum Foundation – Blockchain and smart contract documentation. ethereum.org
8. React Native Documentation – Cross-platform mobile development. reactnative.dev

Experiment VI
ESTIMATION OF PROJECT METRICS

1. COCOMO (Constructive Cost Model)

COCOMO (Constructive Cost Model) is a widely used algorithmic software cost estimation model developed by Barry Boehm. It estimates effort, development time, and team size based on the estimated lines of code (KLOC). For the Jharkhand Eco & Cultural Tourism Platform, the Basic COCOMO Organic model is applied.

1.1 Project Type

Selected Model: Organic Model
Reason for Organic Model Selection:

- Small to medium-sized software project suitable for the Organic category.
- Developed by a small team of 2–4 members with moderate experience.
- Uses modern technologies: React/Next.js (Frontend), Node.js (Backend), MongoDB (Database), Payment Gateway & Blockchain (Security).
- Moderate project complexity involving web/mobile interface, backend REST APIs, AR/VR media hosting, and AI itinerary recommendation engine.
- Team is familiar with the application domain (state-level tourism and cultural booking systems).

1.2 Basic COCOMO Formula (Organic Model)

Effort (E) = a × (KLOC)^b
Time (T) = c × (E)^d
Team Size = E / T

Table 01: COCOMO Organic Model Coefficients

Model	a	b	c	d
Organic	2.4	1.05	2.5	0.38

1.3 Estimated Project Size

Table 02 : Estimated Project Size and Module LOC Distribution

Module	Estimated LOC	Notes
User Authentication & Role-Based Access Control (UC1)	1,000 LOC	Multi-role login (Tourist, LSP, Admin), MFA, sessions
Destination Discovery & Immersive Media (UC2, UC5)	1,500 LOC	Interactive map navigation, geo-fenced content, AR/VR media
Personalized AI Itinerary Planner (UC3)	1,800 LOC	Preference profiling, dynamic scheduling, routing optimization

Multilingual Chatbot & Voice Assistant (UC4)	1,000 LOC	NLP assistant in Hindi, Santhali, Mundari, Ho, English, voice queries
Unified Tourism Booking Engine (UC6)	1,100 LOC	Homestays, tribal tour guides, local events booking, checkout
Tribal Handicrafts & Local Marketplace (UC7)	1,200 LOC	Artisan registration, craft listings, cart, order management
Secure Payment Gateway & Blockchain Ledger (UC8)	1,000 LOC	UPI integration, secure checkout, smart contract log for bookings
Service Provider Verification & Official Analytics (UC9, UC10)	1,400 LOC	LSP registration, PDF certificates, admin dashboard, reports
Database Schema, API Layer & Unified Routing	1,000 LOC	MongoDB collections, backend REST routing, middleware, controllers
Total (Estimated)	~10,000 LOC (≈ 10 KLOC)	

Table 03 : COCOMO Project Metrics Estimation Summary

Metric	**Estimated Value**
Project Size (KLOC)	~10 KLOC (8–12 KLOC range)
Development Effort (E)	~27 Person-Months (range: 18–30)
Development Time (T)	T = 2.5 × (27)^0.38 ≈ 8.7 Months (Math Corrected)
Estimated Team Size	E / T = 27 / 8.7 ≈ 3 Developers

1.4 Estimated Development Effort

Using Basic COCOMO Organic formula with KLOC = 10:

E = 2.4 × (10)^1.05

E = 2.4 × 11.22

E ≈ 26.93 Person-Months

1.5 Resources Required

- Laptop/Desktop (Development machines)
- Stable Internet Connection
- React/Next.js (Frontend development)
- Visual Studio Code (IDE)
- Node.js + Express.js (Backend server)
- MongoDB (NoSQL Database)
- Firebase (Notifications & hosting)
- Git & GitHub (Version control)
- Payment Gateway / Blockchain Ledger API
- i18next Localization Library (Santhali, Mundari, Hindi, English)

1.6 Benefits of Using COCOMO for This Project

• Provides a data-driven estimate of development effort before coding begins.
• Assists in realistic sprint planning and milestone scheduling.
• Predicts the required development time, reducing scope creep risks.
• Helps allocate team members proportionally across modules
• Reduces project planning risks and supports stakeholder communication.

2. Regression Analysis

Regression analysis is a statistical technique used to estimate and predict project metrics such as development effort, project duration, defect density, and resource utilization. It models the linear or non-linear relationship between project size (KLOC) and the development effort required, enabling data-driven forecasting for the Jharkhand Eco & Cultural Tourism Platform project.

2.1 Purpose of Regression Analysis

• Predicts project development effort and resource needs from project size estimates.
• Estimates project completion timelines using historical software project data.
• Identifies trends and anomalies in software development productivity.
• Supports better project planning, risk management, and stakeholder communication.
• Validates and cross-checks COCOMO effort estimates with statistical evidence.

2.2 Regression Variables

Table 04 : Regression Analysis Variables Description

Variable Type	Variable	Description
Independent Variable (X)	Project Size (KLOC)	The estimated number of thousand lines of code in the project.
Dependent Variable (Y)	Development Effort (Person-Months)	The estimated number of person-months required to complete development.

2.3 Sample Dataset for Regression (Reference Projects)

Table 05 : Sample Dataset for Regression Analysis

Project No.	Size X (KLOC)	Effort Y (Person-Months)
P1	2	4.8
P2	4	9.8
P3	6	17.0
P4	8	22.4
P5	10	27.0
P6	12	34.0
P7 (Our Project – Est.)	10	~27

2.4 Linear Regression Model

Regression Line Equation: $Y = a + b \times X$

Where:

a = Intercept (base effort constant) = –2.3

b = Slope (effort per KLOC) = 2.93

Calculation for Our Project (X = 10 KLOC):

Y = –2.3 + 2.93 × 10

Y = –2.3 + 29.3

Y ≈ 27 Person-Months ✓ (Matches COCOMO estimate)

2.5 Regression Graph Description

The scatter plot of Project Size (X-axis, KLOC) vs. Development Effort (Y-axis, Person-Months) shows a clear positive linear trend. As the project size increases from 2 KLOC to 12 KLOC, the effort increases proportionally from approximately 5 to 34 Person-Months. The regression line Y = –2.3 + 2.93X fits closely to the data points, indicating a strong positive correlation ($R^2 \approx 0.98$). Our project falls accurately on the regression line at coordinates (10, 27), validating the COCOMO estimate.

2.6 Expected Outcome of Regression Analysis

• Confirms development effort estimate of approximately 27 Person-Months for 10 KLOC project size.
• Cross-validates COCOMO estimates with data from similar reference projects.
• Provides a quantitative basis for assigning 3–4 team members to complete the project in 7–8 months.
• Enables management to track effort deviations during sprints and adjust resources accordingly.
• Reduces risk of schedule overruns by establishing a data-supported planning baseline.

Conclusion

The COCOMO Organic model estimates that the Smart Digital Platform for Eco & Cultural Tourism in Jharkhand requires approximately 10 KLOC of code, 27 Person-Months of effort, 7–8 months of development time, and a team of 3–4 developers. The Regression analysis independently confirms these estimates, showing a strong linear correlation ($R^2 \approx 0.98$) between project size and effort across reference projects. Together, both models provide a reliable quantitative foundation for project planning, resource allocation, and milestone scheduling.

Project Rationale and Problem Statement

The Jharkhand Eco & Cultural Tourism Platform is proposed to address the following core challenges that currently limit the growth of tourism in the region:

Probable Problems Addressed by the Project

1. Lack of Centralized Digital Tourism Infrastructure: Jharkhand lacks a unified digital platform where tourists can discover destinations, book guides, and plan trips. Information is fragmented across offline sources with no real-time updates.

2. Unorganized and Unverified Local Service Providers: Local guides, homestay hosts, and transport operators operate informally without any standardized verification, making it difficult for tourists to trust and find authentic services.

3. Exclusion of Tribal Artisans from the Digital Economy: Traditional artisans and tribal communities have no digital channel to sell their handicrafts directly to tourists. They rely heavily on middlemen who take large commissions, reducing their income.

4. Insecure and Non-Transparent Payment Systems: Tourists face risks of fraud and overcharging without any payment receipts or secure transaction records. There is no escrow or verification mechanism for tourism-related payments.

5. No Personalized or AI-Based Itinerary Planning: There is no system to provide travellers with custom itineraries based on their interests (adventure, culture, nature, food), duration of stay, and budget, leading to poor tourist experiences.

6. Language and Accessibility Barriers: Most existing tourism apps do not support regional tribal languages such as Santhali, Mundari, and Ho, making them inaccessible to local communities and non-Hindi/English speaking tourists.

7. Difficulty in Estimating Development Effort for Emerging Technologies: The project involves AI engines, blockchain smart contracts, and AR/VR modules — technologies that are complex and difficult to estimate using traditional effort models alone. Regression analysis on historical project datasets is therefore necessary to calibrate COCOMO estimates with actual comparable project data.

Expected Outcomes of Addressing These Problems

- Transparent and verified listings of guides, homestays, and local services.
- Increased income for tribal artisans through direct digital marketplace access.
- Secure blockchain-backed transactions replacing cash-based tourism payments.
- AI-curated travel plans tailored to individual tourist preferences.
- Multilingual interfaces enabling local community participation.
- More accurate project scheduling and cost estimation using regression-calibrated models.

Experiment VII
Modeling Data Flow Diagrams

1. Objective

To design the Context Level (Level 0) and Level 1 Data Flow Diagrams (DFD) for the Jharkhand Eco & Cultural Tourism platform to represent the flow of data between external entities and system processes.

2. Introduction to DFD

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. It models the system's processes, data stores, external entities, and the data flows connecting them. DFDs are hierarchical, starting with the highest level (Context Level) and decomposing into more detailed levels (Level 1, Level 2, etc.).

Key DFD Components:
• External Entities (Rectangles): Represent sources or destinations of data outside the system boundary (e.g., Tourist, Local Service Provider, Tourism Official, Payment Gateway).
• Processes (Circles/Rounded Rectangles): Represent tasks or actions that transform incoming data into outgoing data.
• Data Stores (Open Rectangles/Double Parallel Lines): Represent files or databases where data is stored for later retrieval.
• Data Flows (Arrows): Represent the movement of data between entities, processes, and data stores.

3. Context Level (Level 0) DFD Specifications

The Context Level DFD represents the entire Jharkhand Eco & Cultural Tourism Platform as a single process (Process 0) and maps all inputs and outputs between the system and external entities:
• Tourist: Inputs login credentials, preferences, bookings, payment details, and reviews. Receives search results, AI itineraries, booking confirmations, and payment receipts.
• Local Service Provider: Inputs registration details, handicraft/homestay listings, certifications, and booking updates. Receives booking notifications and payment transfers.
• Tourism Official (Admin): Inputs guide verification approvals, listing validations, and content updates. Receives system analytics and tourism reports.
• Payment Gateway: Inputs transaction authentication and status updates. Receives payment authorization requests.

4. Level 1 DFD Specifications

The Level 1 DFD decomposes the system into 7 core processes and maps them to data stores:
• Process 1.0 (User Management): Tourist/Provider registers and logs in. Interacts with User DB (D1).
• Process 2.0 (Explore & Search): Tourist browses eco-tourism locations, maps, and AR/VR previews. Interacts with Destination DB (D2).

• Process 3.0 (Itinerary Planning): Tourist inputs preferences, AI engine generates itinerary. Interacts with Destination DB (D2).
• Process 4.0 (Booking & Reservation): Tourist books guides/homestays. Interacts with Bookings DB (D3).
• Process 5.0 (Handicraft Marketplace): Providers list and tourists purchase crafts. Interacts with Catalog DB (D4) and Bookings DB (D3).
• Process 6.0 (Secure Payments): Tourist pays, transaction is validated. Interacts with Transactions DB (D5).
• Process 7.0 (Verification & Analytics): Admin verifies guides, views analytics. Interacts with Certifications DB (D6) and all DBs for reports.

5. Data Flow Diagram

5.1 Level 0 — Context Diagram (DFD)

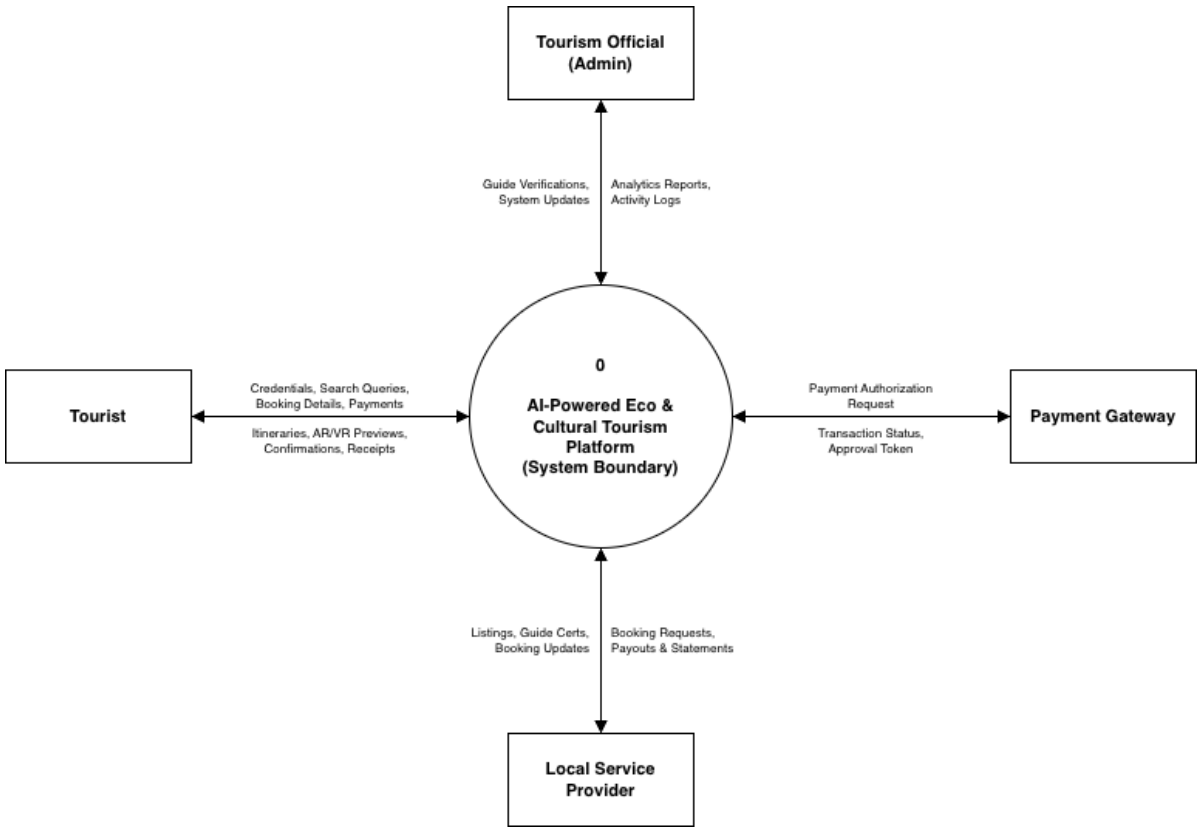


Figure 01 : Level 0 DFD (Context Diagram) for Smart Digital Tourism Platform

5.2 Level 1 — Detailed Process Diagram (DFD)

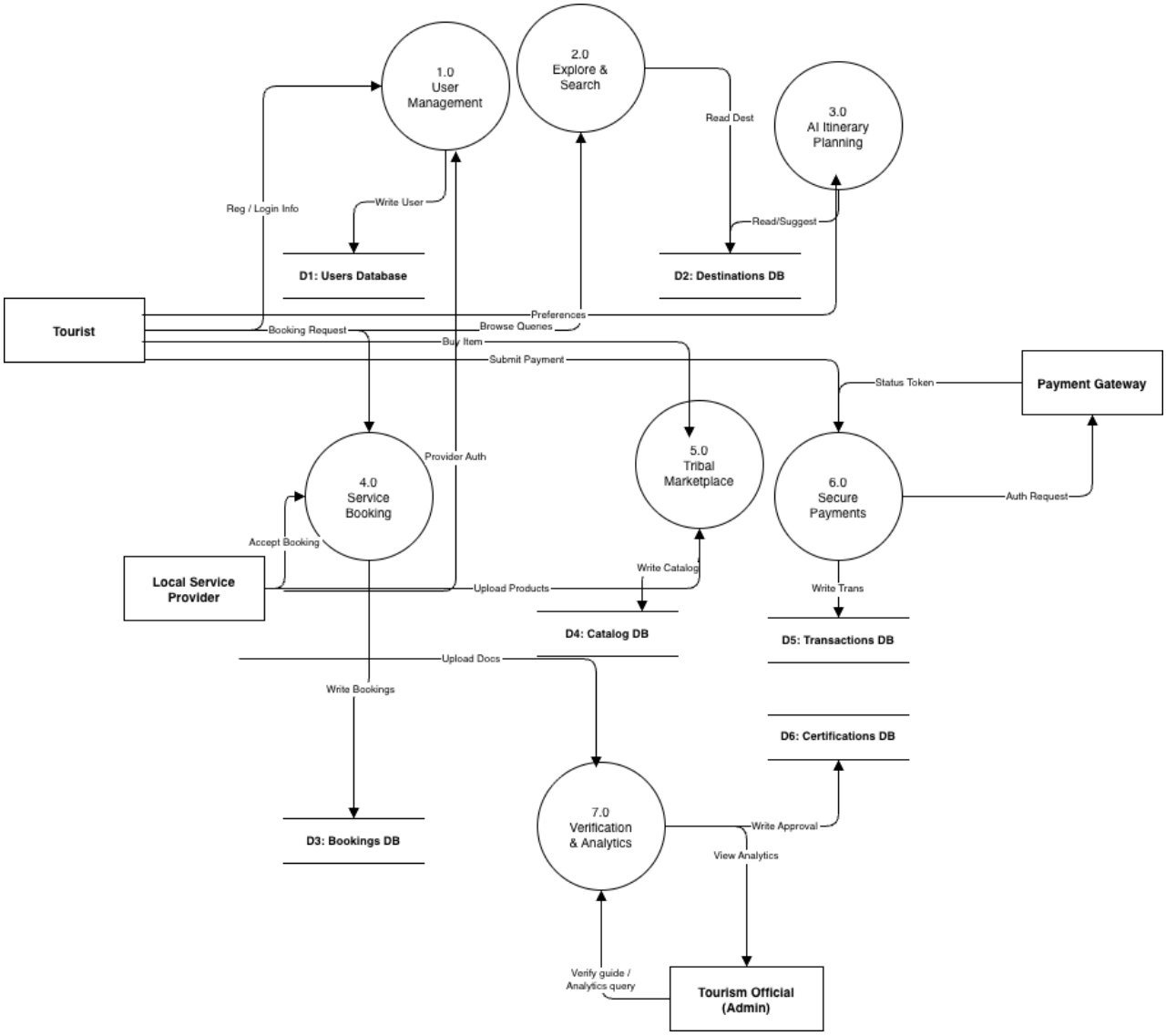


Figure 02 : Level 1 DFD (Detailed Process Diagram) for Smart Digital Tourism Platform

6. Conclusion

The Context Level and Level 1 DFDs successfully map the information flow and data transactions of the Jharkhand Eco & Cultural Tourism Platform, showing how data moves between actors, system processes, and databases.

Experiment VIII

E-R MODELING FROM THE PROBLEM STATEMENTS

Objective

To identify entities, attributes, relationships, and cardinality constraints from the Jharkhand Eco & Cultural Tourism problem statement and model it using an Entity-Relationship (E-R) diagram.

1. Identified Entities and Attributes :

Table 01 : Identified Entities and Attributes for Tourism Platform

Entity Name	Attributes
Tourist	Tourist_ID, Name, Email, Contact_No, Preferences, Password
Local_Service_Provider	Provider_ID, Name, Email, Contact_No, Provider_Type, Verification_Status
Tourism_Official	Admin_ID, Name, Email, Role, Password
Destination	Destination_ID, Name, Location, Description, Popularity_Rating
Booking	Booking_ID, Tourist_ID, Provider_ID, Booking_Date, Total_Amount, Booking_Status
Handicraft_Product	Product_ID, Name, Description, Price, Stock_Quantity, Provider_ID
Payment	Payment_ID, Booking_ID, Transaction_Hash, Amount, Status, Payment_Date
Guide_Certification	Cert_ID, Provider_ID, Certification_Name, Valid_Until, Blockchain_Verified

2. Relationships Between Entities

Table 02: Relationships Between Entities in ER Model

Relationship	Entities Involved	Description
Lists	Local_Service_Provider – Handicraft_Product/Service	Provider lists services (guides, homestays) or marketplace items
Plans	Tourist – Destination	Tourist plans personalized AI itinerary for destinations
Books	Tourist – Booking	Tourist creates bookings for guide or homestay services

Buys	Tourist – Handicraft_Product	Tourist purchases traditional handicrafts on the marketplace
Pays	Tourist – Payment	Tourist makes secure blockchain-enabled payment for services
Verifies	Tourism_Official – Local_Service_Provider Farmer	Officials review and verify provider certifications
Monitors	Tourism_Official – Destination	Officials monitor tourist inflow and popularity trends

3. Cardinality Constraints

Table 03 : Cardinality Constraints Between Entities

Relationship	Cardinality
Provider lists Services/Products	One-to-Many (1:M)
Tourist plans Itinerary for Destination	One-to-Many (1:M)
Tourist places Booking	One-to-Many (1:M)
Tourist buys Handicraft_Product	Many-to-Many (M:N)
Tourist makes Payment	One-to-One (1:1)
Official verifies Local_Service_Provider	One-to-Many (1:M)

4. ER Modeling Diagram

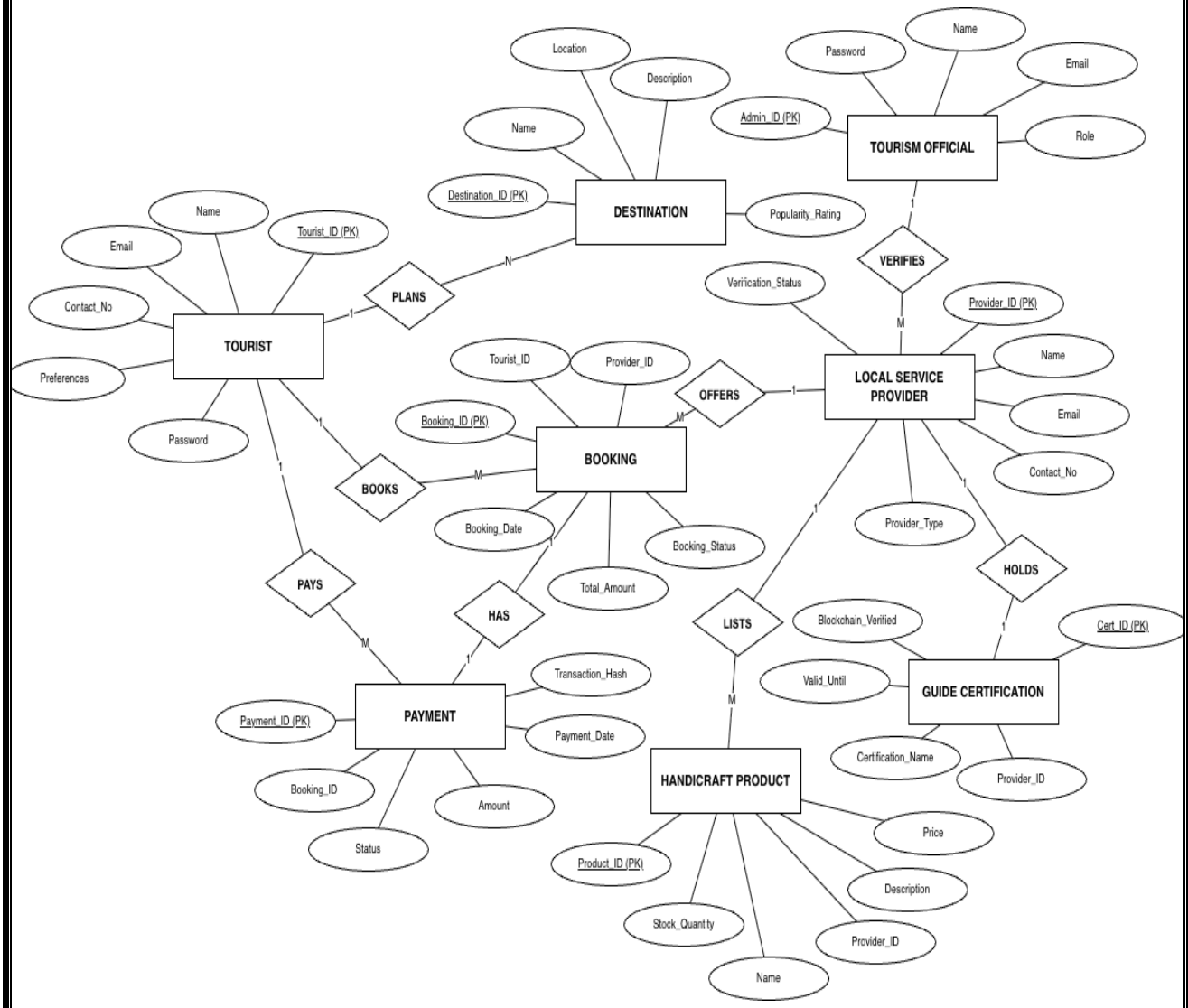


Figure 01 : Entity Relationship Diagram for Smart Digital Tourism Platfor

5. Description of ER Model

The Jharkhand Eco & Cultural Tourism Platform database is designed to manage tourists, local service providers, guide certifications, destinations, bookings, marketplace items, blockchain transactions, and admin records.

The database tracks personalized AI itineraries, guide verifications, booking details, and blockchain-enabled payment transactions to ensure platform trust and reliability.

The E-R model helps in understanding the database structure of the Jharkhand Tourism Platform by clearly defining the tables, columns, relationships, and constraints.

The ER model helps in understanding the database structure of the Farmer Marketplace System by clearly identifying entities, attributes, relationships, and cardinality constraints. It acts as a foundation for database design and helps in developing the system efficiently with minimum data redundancy.

6. Conclusion

The ER model helps in understanding the database structure of the Farmer Marketplace System by clearly identifying entities, attributes, relationships, and cardinality constraints. It acts as a foundation for database design and helps in developing the system efficiently with minimum data redundancy.

Experiment IX
IDENTIFYING DOMAIN CLASSES FROM THE PROBLEM STATEMENT

Problem Statement
The proposed project, Smart Digital Platform to Promote Eco & Cultural Tourism in Jharkhand, provides a centralized platform that connects tourists with local guides, homestay hosts, and tribal artisans. It offers AI itinerary planning, AR/VR previews, multilingual chatbot, guide verification, and blockchain payments.

Step 1: Identify Nouns and Noun Phrases

- Tourist
- Local Service Provider
- Tourism Official (Admin)
- Destination
- Booking
- Handicraft Product
- Payment
- Itinerary
- Guide Certification
- Chatbot Assistant
- AR/VR Preview
- User
- Database
- Mobile Application

Step 2: Categorize the Nouns People

- Tourist
- Local Service Provider
- Tourism Official
- User

Things

- Destination
- Booking
- Handicraft Product
- Payment
- Guide Certification

Concepts

- Multilingual Chatbot
- AR/VR Preview
- Platform Database

Events

- Itinerary Planning
- Service Booking
- Handicraft Purchase

- Blockchain Payment

Step 3: Relevant Domain Classes
After filtering unnecessary terms, the following domain classes are identified:

- Tourist
- Local Service Provider
- Tourism Official
- Destination
- Booking
- Handicraft Product
- Payment
- Guide Certification

Step 4: Additional Domain Classes
Based on the project requirements, a few additional classes can be identified:

- Itinerary
- Itinerary Item
- Blockchain Transaction

Step 5: Generalization

The classes Tourist, Local Service Provider, and Tourism Official share common information such as User ID, Name, Contact Number, Email, and Password. Therefore, we can introduce a generalized superclass called User, and the specific roles will inherit from it.

Final Domain Classes

- User
- User (Superclass)
- Tourist (Subclass)
- Local Service Provider (Subclass)
- Tourism Official (Subclass)
- Destination
- Booking
- Handicraft Product
- Payment
- Guide Certification
- Itinerary
- Itinerary Item
- Blockchain Transaction

Experiment X
MODELING UML CLASS DIAGRAMS

1. Objective

To design the UML Class Diagram for the Jharkhand Eco & Cultural Tourism platform to represent the static structure of the system, including classes, their attributes, methods, and relationships (association, generalization, aggregation, and composition).

2. Introduction to UML Class Diagrams

A UML Class Diagram is a static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects. It is the main building block of object-oriented modeling.

Key Relationships in Class Diagrams:

• Generalization (Inheritance): Represents an 'is-a' relationship. A subclass inherits attributes and methods from a superclass (e.g., Tourist inherits from User).
• Association: Represents a structural relationship between classes where objects of one class are connected to objects of another class (e.g., Tourist is associated with Booking).
• Aggregation: A weak 'has-a' relationship where the child can exist independently of the parent.
• Composition: A strong 'has-a' relationship where the child cannot exist if the parent is destroyed (e.g., Itinerary consists of Itinerary Items).

3. System Class Specifications
The core classes identified for the Jharkhand Eco & Cultural Tourism Platform are specified below with their attributes and operations:

Table 01 : System Class Specifications for Tourism Platform

Class Name	Attributes	Operations (Methods)
User (Superclass)	- userId:String -name:String -email:String - contactNo: String - password: String	+ register(): Boolean + login(): Boolean + updateProfile(): Void
Tourist (extends User)	- preferences: List<String> - languagePreference: String	+ browseDestinations(): List<Destination> + requestItinerary(): Itinerary + makeBooking(d: Booking): Booking + makePayment(bId: String): Payment + submitFeedback(): Void

LocalServiceProvider (extends User)	- providerType: String - verificationStatus: String - documentList: List<String>	+ listService(d: String): Void + listHandicraft(i: Product): Void + acceptBooking(bId: String): Boolean + uploadVerificationDocs(): Void
TourismOfficial (extends User)	- role: String	+ verifyProvider(pId: String): Boolean + generateAnalyticsReport(): Report + configurePlatform(): Void
Destination	- destinationId: String - name: String - description: String - district: String - latitude: Double - longitude: Double - popularityRating: Double	+ getDetails(): Destination + getARVRPreview(): Preview
Booking	- bookingId: String - touristId: String - providerId: String - bookingDate: Date - status: String - totalAmount: Double	+ confirmBooking(): Void + cancelBooking(): Void + getStatus(): String
HandicraftProduct	- productId: String - name: String - description: String - price: Double - stockQty: Int - providerId: String	+ updateStock(qty: Int): Void + getDetails(): Product
Payment	- paymentId: String - bookingId: String - transactionHash: String - amount: Double - status: String - paymentDate: Date	+ processPayment(): Boolean + verifyTransaction(): Boolean

4. Class Diagram

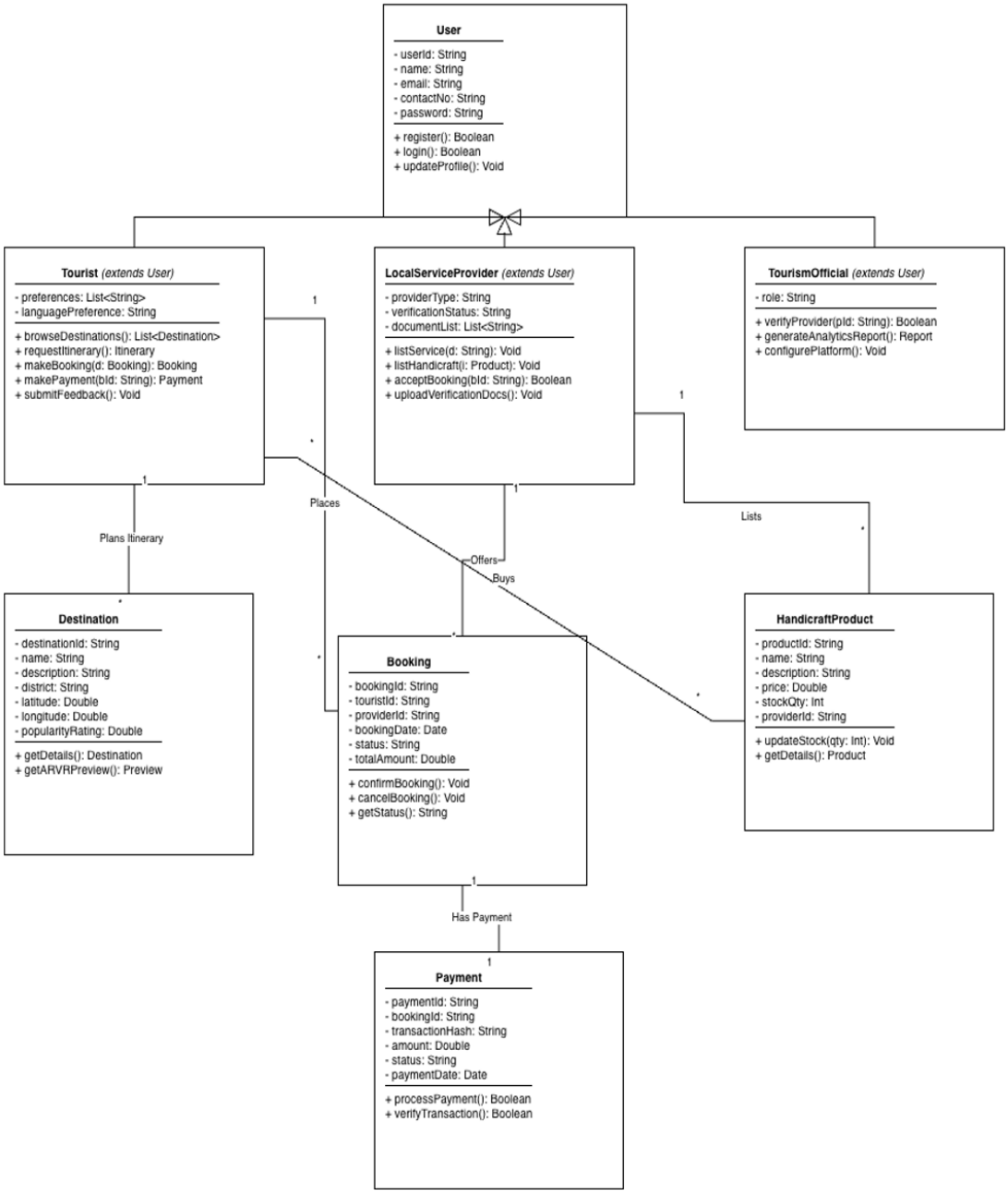


Figure 01: Class Diagram for Smart Digital Tourism Platform

5. Conclusion

The UML Class Diagram models the static structural representation of the Jharkhand Eco & Cultural Tourism Platform, mapping attributes, methods, and relationships (such as the generalization of User types and associations between Tourist, Booking, and Payment).

Experiment XI
MODELING UML SEQUENCE DIAGRAMS

1. Objective

To model UML Sequence Diagrams that capture the dynamic interaction between objects in the Jharkhand Eco & Cultural Tourism Platform, illustrating the time-ordered flow of messages across key system scenarios.

2. Introduction to UML Sequence Diagrams

A UML Sequence Diagram is a type of Interaction Diagram that shows how objects interact in a particular scenario of a use case, emphasizing the time-ordering of messages. It illustrates the sequence of method calls and responses between objects or system components.

Key Components of a Sequence Diagram:

• Lifeline: A vertical dashed line representing an object or actor participating in the interaction.
• Activation Bar: A thin rectangle on a lifeline that shows when an object is executing a method.
• Messages: Horizontal arrows between lifelines representing method calls or signals.
- Synchronous Message (→): The sender waits for a response before continuing.
- Asynchronous Message (--→): The sender does not wait for a response.
- Return Message (--→): A dashed arrow showing the return value from a method.
• Alt Fragment: Combined fragment for conditional / alternative flows (if-else logic).
• Loop Fragment: Combined fragment for repeated interactions.

3. Identified Scenarios

The following three key scenarios have been selected to model the dynamic behaviour of the Jharkhand Eco & Cultural Tourism Platform:

Table 01: Identified Scenarios for Sequence Diagram Modelling

#	Scenario	Actors / Participants	Description
1	Tourist Books a Guide or Homestay	Tourist, TourismApp, AuthService, BookingService, LocalServiceProvider, PaymentService	Tourist logs in, browses destinations, selects a service, confirms booking, and completes blockchain payment.
2	Local Service Provider Lists a Handicraft Product	LocalServiceProvider, TourismApp, MarketplaceService, Database	Provider logs in, fills product details, and system validates and publishes the listing.
3	Tourism Official	TourismOfficial, TourismApp, VerificationService, LocalServiceProvider, Database	Official reviews submitted credentia

	Verifies a Provider		ls, approves/rejects provider, and system notifies the provider.

4. Sequence Diagram Specifications

4.1 Scenario 1: Tourist Books a Guide or Homestay

This sequence captures the complete flow from login to booking confirmation and payment for a tourist using the Jharkhand Tourism Platform.

Table 02 : Tourist Booking and Payment Sequence Flow

Step	From	To	Message / Method	Type
1	Tourist	TourismApp	login(email, password)	Synchronous
2	TourismApp	AuthService	authenticateUser(email, password)	Synchronous
3	AuthService	TourismApp	return authToken	Return
4	TourismApp	Tourist	display: Dashboard	Return
5	Tourist	TourismApp	browseDestinations(filter)	Synchronous
6	TourismApp	BookingService	getAvailableServices(destinationId)	Synchronous
7	BookingService	LocalServiceProvider	checkAvailability(dates)	Synchronous
8	LocalServiceProvider	BookingService	return: available = true	Return
9	BookingService	TourismApp	return: serviceList	Return
10	Tourist	TourismApp	selectService(serviceId, dates)	Synchronous
11	TourismApp	BookingService	createBooking(touristId, serviceId)	Synchronous
12	BookingService	TourismApp	return: bookingId, totalAmount	Return
13	Tourist	TourismApp	initiatePayment(bookingId)	Synchronous
14	TourismApp	PaymentService	processBlockchainPayment(amount)	Synchronous
15	PaymentService	TourismApp	return: txHash, status = SUCCESS	Return
16	TourismApp	Tourist	display: Booking Confirmed!	Return

4.2 Scenario 2: Local Service Provider Lists a Handicraft Product

Table 03 : Handicraft Product Listing Sequence Flow

Step	From	To	Message / Method	Type
1	LocalServiceProvider	TourismApp	login(email, password)	Synchronous

2	TourismApp	AuthService	authenticateUser(email, password)	Synchronous
3	AuthService	TourismApp	return authToken	Return
4	LocalServiceProvider	TourismApp	navigateTo('Marketplace')	Synchronous
5	LocalServiceProvider	TourismApp	submitProductDetails(name, price, stock, images)	Synchronous
6	TourismApp	MarketplaceService	validateAndSaveListing(productDetails)	Synchronous
7	MarketplaceService	Database	insertProduct(productDetails)	Synchronous
8	Database	MarketplaceService	return: productId	Return
9	MarketplaceService	TourismApp	return: listingSuccess	Return
10	TourismApp	LocalServiceProvider	display: Product Listed Successfully!	Return

4.3 Scenario 3: Tourism Official Verifies a Provider

Table 04 : Service Provider Verification Sequence Flow

Step	From	To	Message / Method	Type
1	TourismOfficial	TourismApp	login(email, password)	Synchronous
2	TourismApp	AuthService	authenticateUser(email, password)	Synchronous
3	AuthService	TourismApp	return authToken	Return
4	TourismOfficial	TourismApp	viewPendingVerifications()	Synchronous
5	TourismApp	VerificationService	fetchPendingProviders()	Synchronous
6	VerificationService	Database	getPendingList()	Synchronous
7	Database	VerificationService	return: pendingProviderList	Return
8	VerificationService	TourismApp	return: pendingProviderList	Return
9	TourismOfficial	TourismApp	reviewProvider(providerId)	Synchronous
10	TourismOfficial	TourismApp	approveProvider(providerId)	Synchronous
11	TourismApp	VerificationService	updateStatus(providerId, 'APPROVED')	Synchronous
12	VerificationService	Database	updateProviderStatus(providerId)	Synchronous
13	VerificationService	LocalServiceProvider	notify: 'Account Verified!'	Asynchronous
14	VerificationService	TourismApp	return: updateSuccess	Return
15	TourismApp	TourismOfficial	display: Provider Approved!	Return

5. Sequence Diagram:

5.1 Scenario 1 — Tourist Booking Flow

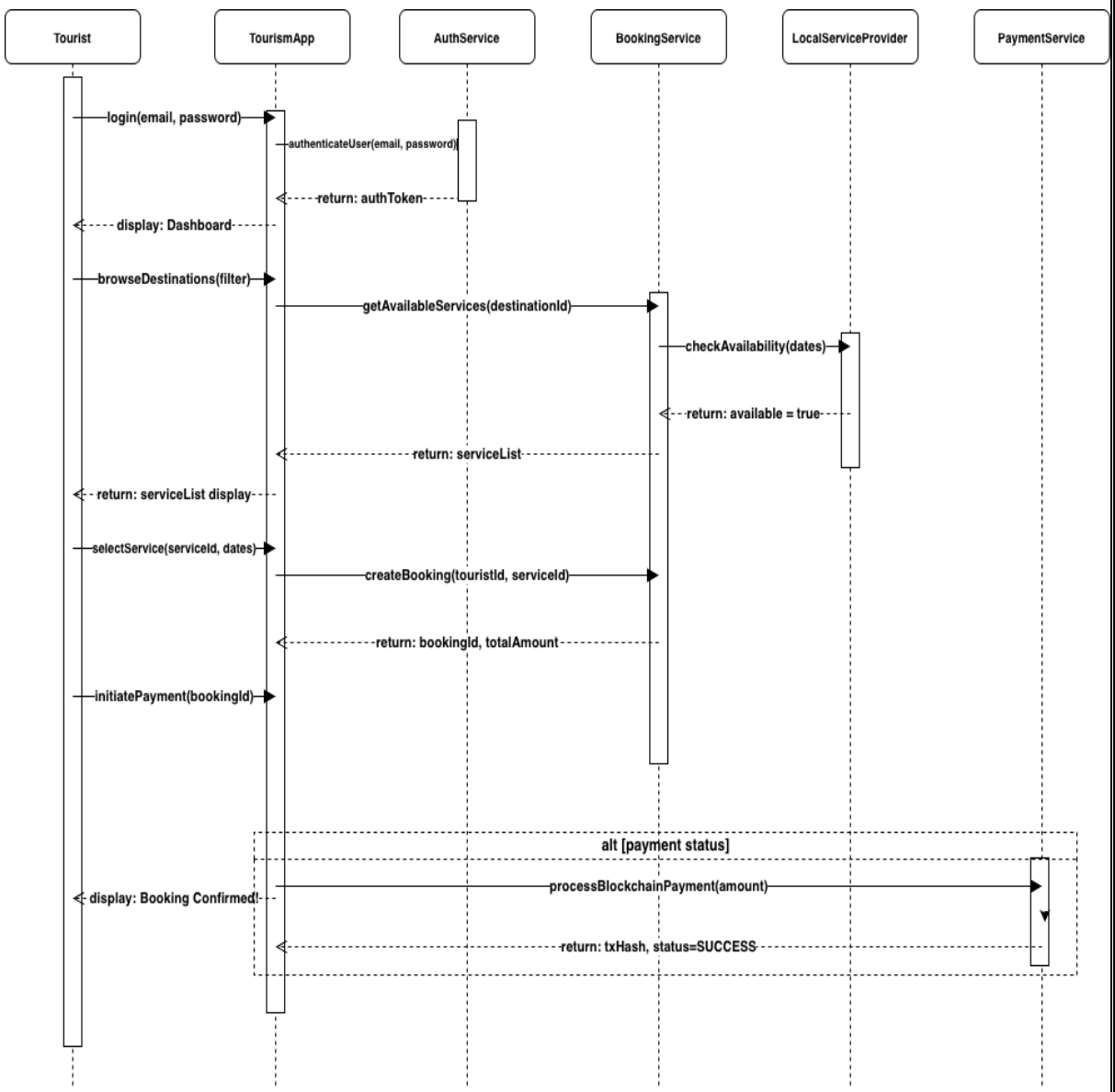


Figure 01 : Tourist Booking and Payment Sequence Diagram

5.2 Scenario 2 — Provider Product Listing Flow

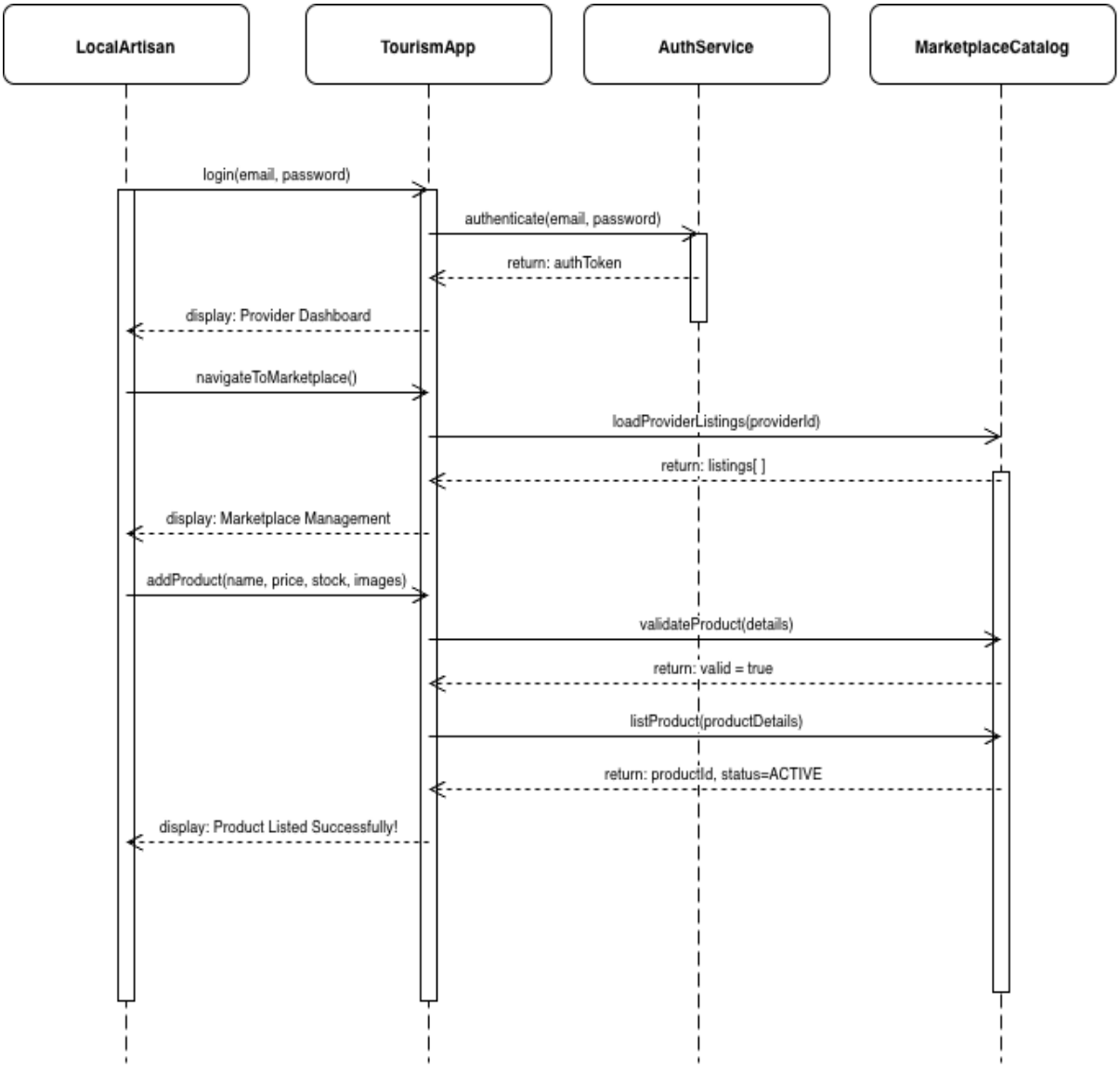


Figure 02 : Handicraft Product Listing Sequence Diagram

5.3 Scenario 3 — Tourism Official Verifies a Provider

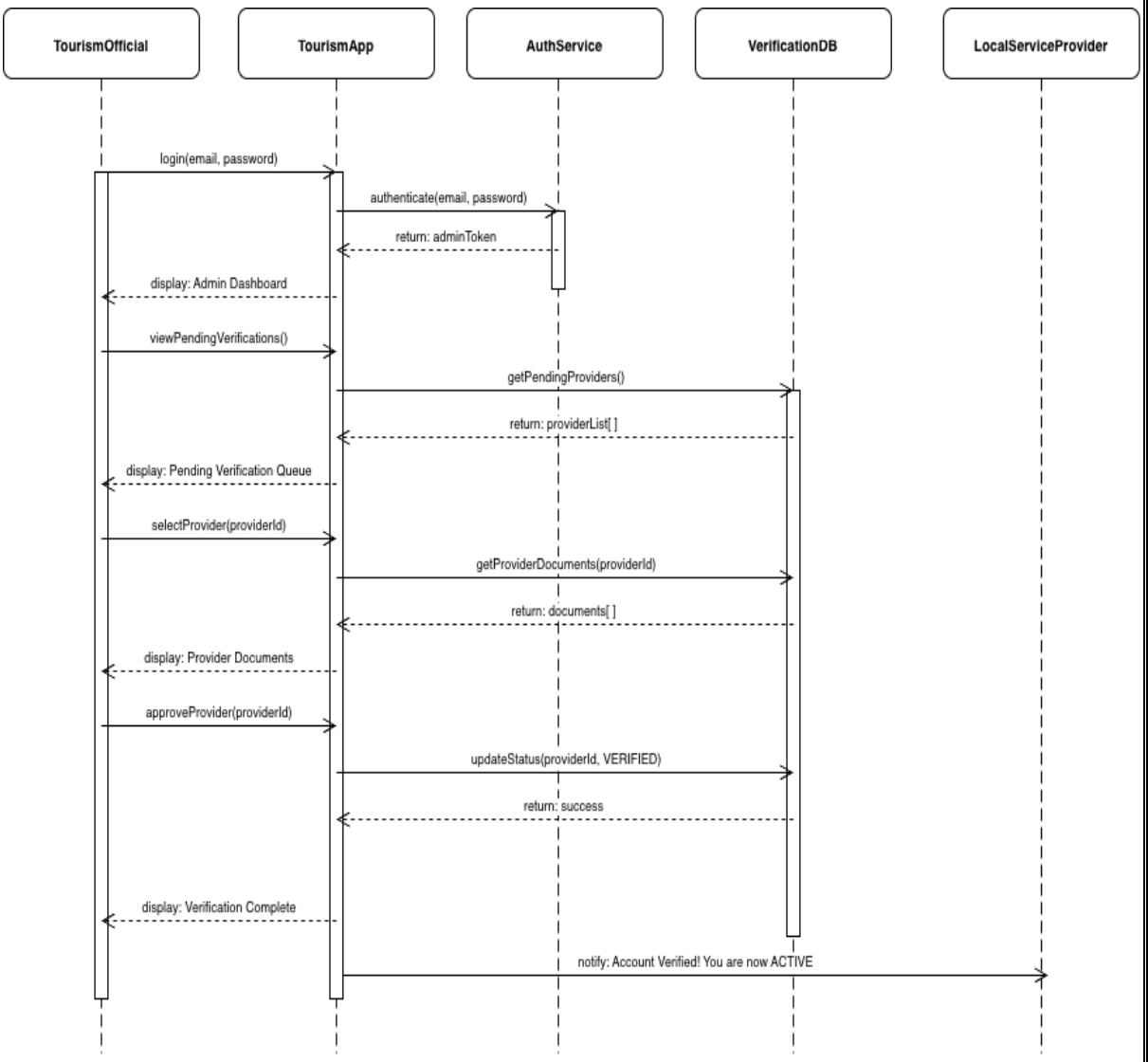


Figure 03 : Service Provider Verification Sequence Diagram

6. Conclusion

The UML Sequence Diagrams for the Jharkhand Eco & Cultural Tourism Platform successfully illustrate the time-ordered flow of messages across three key scenarios: tourist booking, provider product listing, and official provider verification. These diagrams clearly depict how system components interact dynamically, serving as a foundation for detailed design and test case generation.

Experiment XIII
ACTIVITY MODELING

1. Objective

To design the UML Activity Diagrams for the Jharkhand Eco & Cultural Tourism Platform, representing the dynamic procedural workflows, decision paths, and parallel execution steps of core tourist and service provider operations.

2. Introduction to UML Activity Diagrams

A UML Activity Diagram is essentially a flowchart that represents the flow of control from one activity to another within a system. It is used to model business processes, system use cases, and complex algorithmic flows, showing both sequential and concurrent actions.

Key Elements of Activity Diagrams:

• Start Node (Solid Black Circle): Indicates the beginning of the workflow.
• Activity/Action (Rounded Rectangle): Represents a step or executable action within the process.
• Control Flow (Solid Arrow): Indicates the transition direction of control.
• Decision Node (Diamond): A point in the workflow where control branches into alternative paths based on guard conditions.
• Merge Node (Diamond): Merges multiple alternative control paths back into a single stream.
• Fork Bar (Solid Thick Line): Splits a single control path into multiple parallel (concurrent) activities.
• Join Bar (Solid Thick Line): Synchronizes multiple concurrent activities back into a single control flow.
• End Node (Solid Circle with Ring): Marks the termination of the entire workflow.
• Swimlanes: Vertical columns that organize activities according to the actor or system component responsible for them.

3. Workflows Selected for Activity Modeling

The following two operational workflows have been mapped to visualize their process logic and concurrency rules:

Table 01:Workflow Selection for Activity Modeling

Workflow ID	Workflow Name	Responsible Actors	Core Objective
WF-01	Tourist Booking and Secure Payment Process	Tourist, Tourism Platform, Payment Gateway, Local Provider	Flow of exploring eco-tourism destinations, checking booking slots, performing conditional blockchain payment checks, and concurrently notifying stakeholders.

WF-02	Handicraft Product Listing and Purchase Cycle	Local Artisan, Platform Marketplace, Tourist, Shipping/Logistics	Flow of uploading artisan crafts, validating stocks, enabling buyer checkout, and updating ledger logs for marketplace sales.

4. Activity Workflow Specifications

4.1 Workflow 1: Tourist Booking and Payment Flow Steps

Table 02: Tourist Booking and Secure Payment Workflow Steps

Step No.	Actor	Action Description	Control Flow / Next Step
1	Tourist	Selects a destination and inputs booking dates.	Goes to Step 2 (Check slots).
2	Tourism Platform	Checks availability of guides/homestays in database.	Decision: If slot is free, go to Step 3. If full, notify Tourist and exit.
3	Tourist	Confirms booking and initiates checkout.	Goes to Step 4 (Process Payment).
4	Payment Gateway	Processes blockchain transaction request.	Decision: If successful, go to Step 5. If failed, prompt for retry or cancel.
5	Tourism Platform	Forks the workflow into two parallel threads.	Initiates concurrent execution of Steps 6 and 7.
6	Tourism Platform	Sends transaction receipt and booking confirmation to Tourist.	Merges at Join Bar.
7	Tourism Platform	Dispatches booking notification to Local Service Provider.	Merges at Join Bar.
8	Tourism Platform	Joins parallel flows and records confirmation in blockchain ledger.	Ends workflow.

4.2 Workflow 2: Handicraft Listing and Purchase Cycle Steps

Table 03 : Handicraft Listing and Purchase Cycle Workflow Steps

Step No.	Actor	Action Description	Control Flow / Next Step
1	Local Artisan	Inputs craft details (name, price, stock, description).	Goes to Step 2.
2	Platform Marketplace	Validates listing details and checks image guidelines.	Decision: If approved, go to Step 3. If rejected, request re-upload.
3	Platform Marketplace	Publishes listing and updates active inventory.	Goes to Step 4 (Tourist Browses).
4	Tourist	Adds handicraft item to marketplace shopping cart and clicks checkout.	Goes to Step 5.
5	Platform Marketplace	Deducts quantity from stock and locks item temporary.	Goes to Step 6 (Payment processing).
6	Payment Gateway	Executes transaction verification.	Decision: If successful, go to Step 7. If failed, restore stock quantity and exit.
7	Platform Marketplace	Forks flow: Notifies Artisan for shipping and updates public ledger record.	Joins parallel threads and ends workflow.

5. Activity Diagrams

5.1 Workflow 1 Diagram — Tourist Booking and Payment Process :

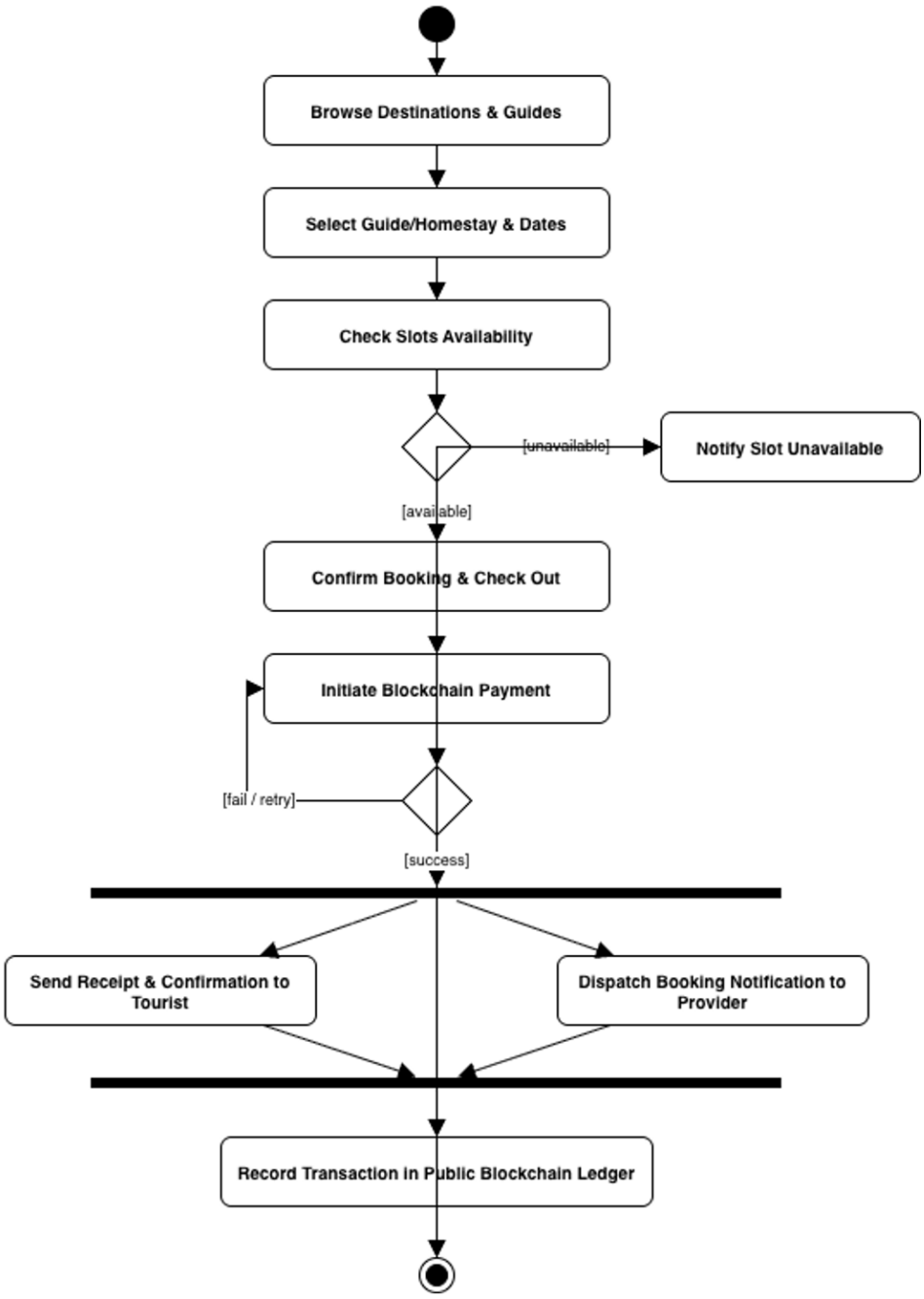


Figure 01: Tourist Booking and Payment Activity Diagram

5.2 Workflow 2 Diagram — Artisan Handicraft Listing and Purchase Flow

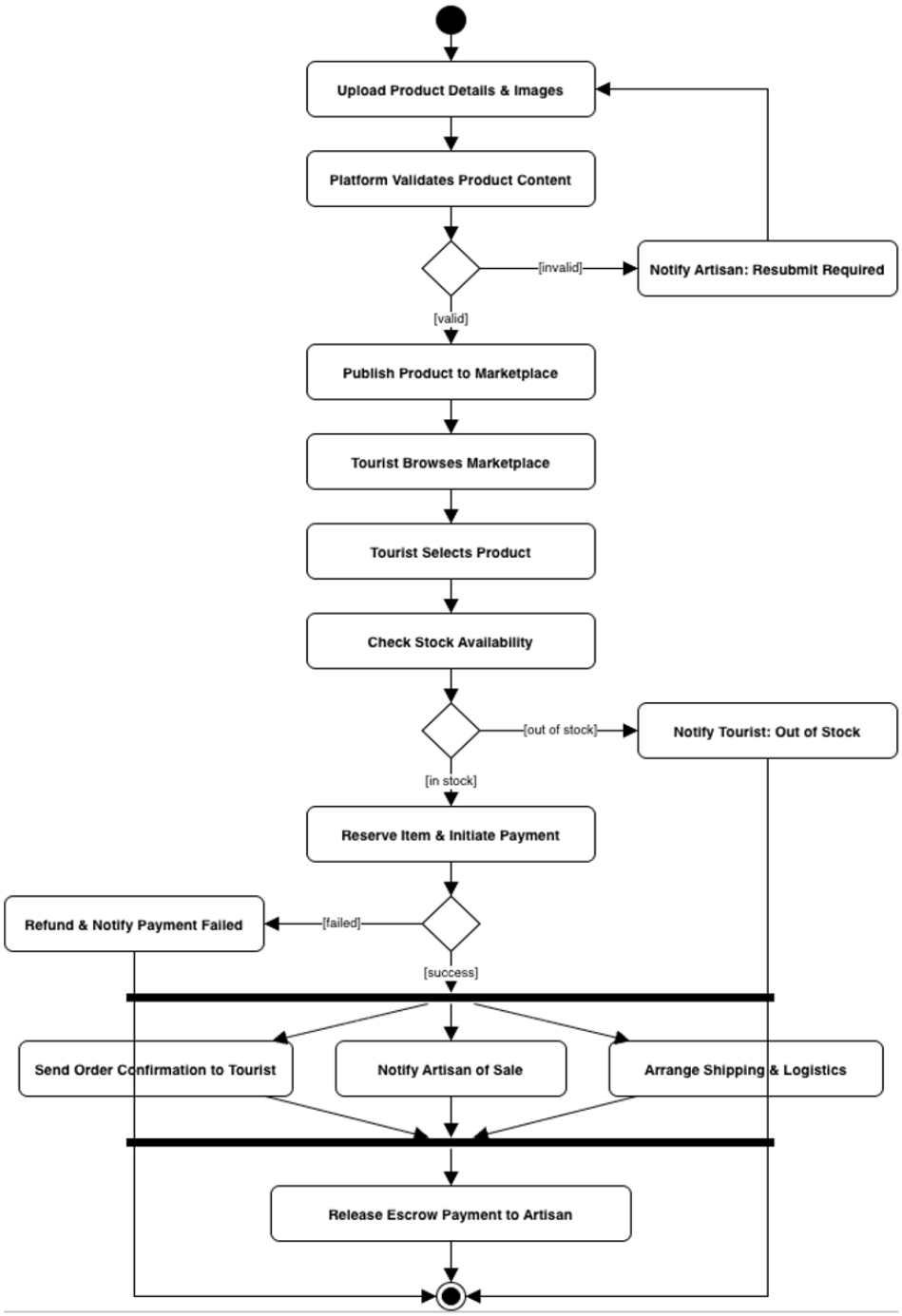


Figure 02: Handicraft Listing and Purchase Cycle Activity Diagram

6. Conclusion

The UML Activity Diagrams successfully capture the procedural flow, decisions, and parallel processing capabilities of the Jharkhand Eco & Cultural Tourism Platform. By defining clear actors in swimlanes and mapping concurrency via forks and joins, these diagrams clarify the end-to-end execution paths, assisting programmers in implementing error-free transaction workflows.

Experiment XVI
SOFTWARE REQUIREMENT SPECIFICATION (SRS)

AI-Powered Smart Digital Platform to Promote Eco & Cultural Tourism in Jharkhand

Table of Contents

1. Introduction
1.1 Purpose
1.2 Document Conventions
1.3 Intended Audience and Reading Suggestions
1.4 Product Scope
1.5 References
2. Overall Description
2.1 Product Perspective
2.2 Product Functions
2.3 User Classes and Characteristics
2.4 Operating Environment
2.5 Design and Implementation Constraints
2.6 User Documentation
2.7 Assumptions and Dependencies
3. External Interface Requirements
3.1 User Interfaces
3.2 Hardware Interfaces
3.3 Software Interfaces
3.4 Communications Interfaces
4. System Features
5. Other Nonfunctional Requirements
6. Other Requirements & Conclusion

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the AI-Powered Smart Digital Platform to Promote Eco & Cultural Tourism in Jharkhand (Version 1.0), developed under the guidelines of the Smart India Hackathon (SIH). The product targets the Ministry of Tourism, Government of India, and the Department of Tourism, Government of Jharkhand. This document covers the full scope of the web and mobile-responsive application ecosystem, including the tourist-facing recommendation and booking portal, the local service provider management console, and the administrative back-office verification panel. Version 1.0 encompasses the initial release of all core modules.

1.2 Document Conventions

The following conventions apply throughout this document:

- Bold text denotes section headings, key terms, and table column headers.
- REQ-F-XXX denotes functional requirements; REQ-NF-XXX denotes non-functional requirements.
- Priority levels are: High (H), Medium (M), Low (L).
- TBD = To Be Determined; these items are catalogued in Appendix C.
- All requirements are stated in the present tense and use 'shall' for mandatory items.

1.3 Intended Audience and Reading Suggestions

This document is intended for the following stakeholders:

- Developers & Architects – Read Sections 3, 4, and 5 for interface and feature specifications.
- Project Managers – Focus on Section 2 (product overview) and Section 5 (non-functional requirements).
- QA / Testers – Refer to Section 4 (system features and REQ IDs) for test case derivation.
- Government / Department Officials (Ministry of Tourism) – Review Sections 1, 2, and 6 for policy compliance and business metrics.
- End Users (Tourists / Local Service Providers) – Section 2.3 describes roles and system interaction models.

New readers should begin with Section 1.4 (Product Scope) and Section 2 for a broad system overview before diving into detailed requirements in Sections 3–5.

1.4 Product Scope

The AI-Powered Smart Digital Platform to Promote Eco & Cultural Tourism in Jharkhand is a comprehensive digital ecosystem designed to bridge the gap between Jharkhand's rich, unexplored natural/cultural heritage and the modern digital traveler. The system addresses the issues of manual, unorganized, and fragmented tourism discovery by implementing:

- AI-based customized itinerary planning based on user preference tags.
- Multilingual chatbot assistance for navigation and local guidance.
- Verified registries for local guides and homestay hosts using secure ID uploading.
- A decentralized blockchain-based ledger for escrow payment settlements and credential checks.
- An online direct-to-consumer marketplace for tribal artisans to sell local handicrafts.

The system directly supports the government's objectives of promoting local livelihoods, preserving tribal heritage, and encouraging sustainable eco-tourism.

1.5 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- Smart India Hackathon (SIH 2024/2025) Problem Statement — Ministry of Tourism, Govt of India.
- Jharkhand Tourism Policy Guidelines (Government of Jharkhand).
- Mapbox API Documentation — docs.mapbox.com
- Web3.js / Ethereum Developer Documentation — web3js.readthedocs.io
- OpenAI / LangChain API Reference Guide.

2. Overall Description

2.1 Product Perspective

The Jharkhand Eco & Cultural Tourism Platform is a self-contained, full-stack digital ecosystem initiated to complement the existing tourism infrastructure of Jharkhand. It is not a replacement of any existing system but a greenfield solution designed to digitize and intelligently automate the local tourism economy.

The system consists of three major subsystems communicating over secure REST APIs:

- Tourist Portal — A web and mobile-responsive application for tourist registration, preference selection, itinerary generation, marketplace browsing, and guide booking.
- Service Provider Portal — A portal for local guides, homestay hosts, and tribal artisans to register, manage listings, upload verification documents, and process orders.
- Admin Dashboard — An administrative console for tourism officials to verify service providers, view platform-wide analytics, and audit security logs.

2.2 Product Functions

The major functional capabilities of the platform are:

- User Registration & Verification — Aadhaar/ID upload and verification for local providers.
- AI-Powered Itinerary Planning — Custom travel routes based on user tags.
- Guide & Homestay Booking — Real-time slot availability check and instant booking.
- P2P Artisan Marketplace — Direct handicraft sales with secure online checkout.
- Escrow Payments — Blockchain-secured holding of funds until service delivery.
- Real-Time Guide Tracking — Location sharing for active tourist bookings.
- Multilingual Support — Platform accessibility in Hindi, English, Santhali, Mundari, and Ho.
- Admin Analytics — Charts and reports on registrations, revenue, and verification queues.

Table 01: 2.3 User Classes and Characteristics

User Class	Description & Characteristics
Tourist	Casual travelers looking for offbeat destinations. Require simple, responsive web/mobile interfaces. High frequency of use during travel planning and actual trips.
Local Service Provider (LSP)	Local guides, homestay hosts, and tribal artisans. May have moderate digital literacy. Require mobile-first, low-bandwidth interfaces with voice-assist. Daily frequency of use.

Tourism Official (Admin)	Government/department administrators. High technical and regulatory proficiency. Access via secure dashboard to verify credentials and audit logs. Periodic frequency of use.

2.4 Operating Environment

- Client-Side: Modern browsers (Chrome 110+, Safari 15+, Edge 110+), Android 9.0+, iOS 14+ mobile devices.
- Server-Side: Ubuntu 22.04 LTS on cloud infrastructure (AWS / Microsoft Azure / GCP).
- Backend Framework: Python (FastAPI / Django REST Framework), Node.js for real-time notifications.
- Database: PostgreSQL 15 (relational), Redis (session caching and rate limiting).
- AI/ML Runtime: Python 3.10+ with PyTorch/TensorFlow for model serving.
- Blockchain: Ethereum private ledger / Hyperledger Fabric network.

2.5 Design and Implementation Constraints

- Regulatory Compliance: Data storage must comply with the IT Act 2000 and DPDP Bill 2023.
- Blockchain Gas Optimization: Escrow transactions must minimize execution costs for micropayments.
- Accessibility: UI must comply with WCAG 2.1 Level AA standards.
- Offline Support: Essential mapping and itinerary details must be cached offline due to poor network coverage in remote zones.

2.6 User Documentation

- Tourist User Manual — In-app tutorial and PDF guide for search, bookings, and payments.
- LSP Listing Guide — Visual guides and local-language video walkthroughs for listing services/products.
- Admin Operations Manual — Guide on verifying documents, reading charts, and resolving disputes.

2.7 Assumptions and Dependencies

Assumptions:

- Users have basic GPS-enabled smartphones.
- LSPs possess valid government identity proofs.
- Stable internet connection exists at tourist destinations.

Dependencies:

- Mapbox API availability for routing and mapping functions.
- Stable payment gateways for fiat transactions.
- Ethereum private node connectivity for smart contract executions.

3. External Interface Requirements

3.1 User Interfaces

- Tourist Dashboard: Simple card layout showing recommended activities, saved guides, and active itineraries.
- Artisan Storefront: Visual grid layout for local handicrafts with 'Add to Cart' and 'Checkout' actions.

• Map Interface: Integrated map interface displaying nearby attractions, homestays, and active guide paths.
• Admin Console: Tabbed layout for verification queues, active listings, and user logs.

All interfaces shall follow the National Government Design Standards (NGDS) for Indian e-governance portals.

3.2 Hardware Interfaces

• GPS Module: Accessed via browser geolocation APIs for navigation and location sharing.
• Camera/Scanner: Managed via mobile browser capture APIs for guide certificate uploads.

3.3 Software Interfaces

• PostgreSQL 15: Relational database for accounts, bookings, and product catalogs.
• Mapbox SDK: For routing algorithms and interactive mapping.
• Web3.js: To interface with private smart contracts for escrow.
• OpenAI API: For processing tourist preference vectors.

3.4 Communications Interfaces

• All client-server traffic encrypted via TLS 1.3 (HTTPS).
• WebSockets for instant messaging between tourists and guides/chatbot.
• Transactional SMS/Email notifications sent via HTTP POST to external gateway APIs.

4. System Features

4.1 Feature F-01: User Registration & Identity Verification

4.1.1 Description and Priority

Allows tourists to sign up and local providers to submit KYC credentials. Priority: High.

4.1.2 Stimulus / Response Sequences

• LSP uploads Aadhaar/PAN -> System encrypts and stores files -> Status set to 'Pending Verification' -> Admin receives verification task.

4.1.3 Functional Requirements

REQ-F-001: The system shall permit tourists to register via email or mobile number with OTP verification.
REQ-F-002: The system shall require Local Service Providers (LSPs) to upload identification documents (Aadhaar/PAN/Guide License) in PDF or PNG format up to 4 MB.
REQ-F-003: The system shall maintain an LSP's account status as 'Pending' and disable public visibility of their listings until approved by an Admin.
REQ-F-004: The system shall reject duplicate registrations matching an already registered Aadhaar or email address.

4.2 Feature F-02: AI-Powered Custom Itinerary Planner

4.2.1 Description and Priority

Generates customizable, geographically optimized tour schedules. Priority: High.

4.2.2 Stimulus / Response Sequences

• Tourist selects interest tags (e.g., Waterfalls, Tribal Culture) and duration -> AI engine generates customized schedule -> Tourist edits and saves itinerary.

4.2.3 Functional Requirements

REQ-F-005: The system shall capture tourist preferences including duration (1–10 days) and interest tags (Eco-tourism, Tribal Culture, Waterfalls, Historical Temples).
REQ-F-006: The system shall call the OpenAI API/LangChain model to generate a sequential day-wise travel plan optimizing geographic distance between destinations.
REQ-F-007: The system shall display the generated route on an interactive Mapbox map with navigation indicators.
REQ-F-008: The system shall allow tourists to manually add, swap, or delete destinations in the itinerary and recalculate travel times.

4.3 Feature F-03: AI-Based Local Service & Product Recommendation Engine

4.3.1 Description and Priority

Matches tourists with guides/homestays and tribal products. Priority: High.

4.3.2 Stimulus / Response Sequences

• Tourist views destination -> Recommendation engine displays high-rated local guides and nearby homestays -> Tourist views matching artisan handicrafts.

4.3.3 Functional Requirements

REQ-F-009: The system shall rank and recommend registered local guides within 25 km of the tourist's planned destinations based on experience and ratings.
REQ-F-010: The system shall recommend verified homestays matching the tourist's budget and group size.
REQ-F-011: The system shall suggest relevant tribal handicraft products relevant to the tourist's current location.
REQ-F-012: Recommendation indexes shall be refreshed within 1 hour of a guide updating their status.

4.4 Feature F-04: Guide/Homestay Booking & Real-Time Tracking

4.4.1 Description and Priority

Manages slot reservation and GPS tracking. Priority: High.

4.4.2 Stimulus / Response Sequences

• Tourist checks slots -> Blocks slot -> Pays -> System starts booking -> Prompts guide.

4.4.3 Functional Requirements

REQ-F-013: The system shall display calendar slot availability for verified guides and homestays.
REQ-F-014: The system shall lock booking slots for 10 minutes during payment checkout.
REQ-F-015: The system shall allow tourists to track the guide's live location on the map during the booked duration.

REQ-F-016: The system shall send real-time check-in/check-out status updates.

4.5 Feature F-05: Local Service Provider Listing & Order Management

4.5.1 Description and Priority
Enables guides, hosts, and artisans to manage offerings. Priority: High.

4.5.3 Functional Requirements
REQ-F-017: The system shall allow approved guides to create custom packages with prices, languages, and slots.
REQ-F-018: The system shall allow artisans to list handicraft items with images, descriptions, and stock counts.
REQ-F-019: The system shall provide LSPs with a dashboard to view active orders and booking history.
REQ-F-020: The system shall notify LSPs of new orders via push notifications and SMS.

4.6 Feature F-06: Tourism Department Admin Analytics & Verification Dashboard

4.6.1 Description and Priority
Oversight tool for administrative compliance. Priority: Medium.

4.6.3 Functional Requirements
REQ-F-021: The dashboard shall display metrics: total active listings, verification backlogs, and total transaction volume.
REQ-F-022: The system shall generate weekly compliance reports in PDF/Excel formats.
REQ-F-023: The system shall allow admins to revoke LSP approvals for policy violations with a documented log.

5. Other Nonfunctional Requirements

5.1 Performance Requirements
REQ-NF-001: The dashboard page shall load in under 2 seconds on a 3G mobile connection (>= 2 Mbps).
REQ-NF-002: AI-based itinerary queries shall return results within 4 seconds of request submission.
REQ-NF-003: The system shall support 5,000 concurrent active users without slow-down.
REQ-NF-004: Database response times for booking checkouts shall not exceed 300ms.

5.2 Safety Requirements
REQ-NF-005: The mobile application shall include a one-tap SOS button that sends the tourist's current GPS location to the nearest police station.
REQ-NF-006: System databases must perform automated nightly backups to secure, redundant cloud storage.
REQ-NF-007: If the payment network fails, the booking status shall rollback to 'Pending Payment' to prevent double-charging.

5.3 Security Requirements

REQ-NF-008: All user credentials and passwords shall be hashed using bcrypt (cost factor 12).
REQ-NF-009: All PII (Aadhaar, phone numbers) must be stored in encrypted form using AES-256.
REQ-NF-010: The system shall implement CSRF tokens, secure session cookies, and SQL-injection filters.
REQ-NF-011: User sessions (JWT tokens) shall expire after 30 minutes of inactivity.
REQ-NF-012: All admin actions (approvals/revocations) must be logged in a tamper-evident database audit trail.

Table 02: 5.4 Software Quality Attributes

Quality Attribute	Specification
Availability	99.9% uptime SLA; scheduled maintenance under 2 hours/month.
Scalability	Autoscale server instances when CPU usage exceeds 70%.
Maintainability	Microservice structure separating AI, Booking, and Marketplace services.
Usability	Booking process completed in less than 3 minutes.
Portability	Fully containerized (Docker) for multi-cloud deployment.
Reliability	MTTR less than 20 minutes for system failures.

5.5 Business Rules

- Only guides verified by Jharkhand Tourism department can publish public bookings.
- Booking escrow funds are released to providers only after tourist check-out confirmation or 24 hours post-tour completion.
- Artisan handicraft listings must undergo a content-moderation check.
- Booking cancellations within 24 hours of start time receive a maximum of 50% refund.

6. Other Requirements & Conclusion

- Database daily backups at 03:00 AM IST.
- Multilingual support for Jharkhand local dialects (Santhali, Mundari, Ho, Hindi) and English.

6.1 Conclusion

This Software Requirement Specification (SRS) serves as the definitive reference document for the development, testing, and deployment of the Jharkhand Eco & Cultural Tourism Platform. It establishes a common agreement between all stakeholders, ensuring that the final software product meets the standards and requirements outlined during the engineering lifecycle.

Acropolis Institute of Technology and Research Indore

Department of Computer Science and Engineering

Industry Problem Identification Activity Assignment

(Jan-June 2026)

1. Overview

- This is a field-based industry interaction activity designed for students to visit nearby companies in their surroundings to identify real-world problems.
- The main focus is on finding genuine, impactful, and technically feasible real time problems that can be solved using Computer Science & Engineering concepts and technologies.

2. Objective of the activity

- To bridge the gap between academic knowledge and real industry requirements.
- To develop problem identification, requirement analysis, and industry interaction skills.
- To ensure capstone projects are relevant and solve actual problems.
- To build strong industry connections.
- To understand the working environment of the industry.
- To identify practical real-time challenges faced by the industries.
- To critically analyse the root causes of the identified problem.
- To document and report the problem systematically for future reference and project implementation.

3. Assignment Requirements

- The activity is to be carried out by the **team of 2 students.**
- Students must select one organization/industry from nearby surrounding areas (Pithampur, Dewas, Sanwer, etc.).
- Students team will visit the selected industry to identify real-world problems that can be solved by the intervention of computers.
- Preferred Industry sectors: Manufacturing, Healthcare, Logistics, Education, Retail, Finance, Start-ups, E-commerce, etc.

PART A: Industry Details

1. Industry/Organization Name: Riddhi Siddhi Honda Showroom & Service Center

2. Industry Type/Domain: Automobile Retail & Vehicle Service Industry

3. Address: 2, Sanwer Road, Near Aurobindo Hospital, Bhawrasla, Indore, Madhya Pradesh – 453555

4. Contact Person Name: Mohit Hirwe

5. Designation: Mechanic

6. Contact Details (Phone & Email): Phone: 7470973887 | Email: mohithirwe123@gmail.com

7. Date of Visit: 08 July 2026

PART B: Problem Identified

1. Problem Title: Inefficient Vehicle Service Tracking, Customer Record Management and Spare Parts Monitoring System

2. Problem Category: Data Handling / Management / Customer Service / Automation

3. Detailed Problem Description:

The Honda showroom and service center manages a large number of customers daily for bike purchases, servicing, maintenance, repairs, and spare part replacements. Proper management of customer information, vehicle details, previous service records, warranty details, and maintenance schedules is very important for smooth operation.

During interaction with the service center staff, it was observed that managing and accessing complete vehicle service history, customer follow-ups, and spare parts information can become difficult when customer volume increases.

The issue mainly occurs during servicing operations when employees need to check previous repairs, maintenance history, and availability of required spare parts. This affects service staff, mechanics, and customers as it increases dependency on manual checking and communication.

4. Existing System / Current Process:

Currently, when a customer brings a vehicle for servicing, staff members collect customer details, inspect the vehicle, check service requirements, and maintain service-related information through existing records and communication methods.

Mechanics perform vehicle inspection, identify required repairs, check spare parts availability, and update the customer about service progress manually. Customer reminders and follow-ups are mostly handled through direct communication or phone calls.

5. Issues / Challenges in Current Process:

- Difficulty in quickly accessing complete vehicle service history
- More time required to track previous repairs and maintenance records
- Possibility of missing scheduled service reminders
- Difficulty in managing spare parts availability information
- Increased workload during high customer visits
- Manual communication causes delays in service updates
- Generating service reports and analyzing customer data becomes time-consuming
- Tracking repeat customers and their vehicle problems is difficult

6. Impact of the Problem:

The problem affects the overall efficiency of the service center by increasing the time required for record searching and service management. Employees need additional effort to maintain customer information and provide updates.

Customers may experience longer waiting times during busy periods. Lack of quick access to organized service history can reduce service speed, customer satisfaction, and overall productivity of the business.

7. Proposed Solution (CS Perspective):

As Computer Science students, this problem can be solved by developing a Digital Vehicle Service Management System.

The proposed solution includes:

- **Customer & Vehicle Database:** A centralized database to store customer details, vehicle model, registration number, service history, warranty information, and repair records.
- **Service Tracking System:** A module where employees can update vehicle service progress, completed repairs, and delivery status.
- **Automated Service Reminder:** The system can automatically notify customers about upcoming servicing dates and maintenance requirements.
- **Spare Parts Inventory Management:** Tracks available spare parts, used items, and alerts employees when stock becomes low.
- **Digital Billing and Reports:** Generates digital invoices and daily/monthly reports for better business analysis.
- **Admin Dashboard:** Allows managers to monitor service requests, customer records, employee work, and overall operations.

This system will improve data accuracy, reduce manual work, save time, and enhance customer satisfaction.

8. Proof of Visit

Photographs from the visit to Riddhi Siddhi Honda Showroom & Service Center, Indore (08 July 2026):

Figure 1: Visit to Riddhi Siddhi Honda Showroom & Service Center

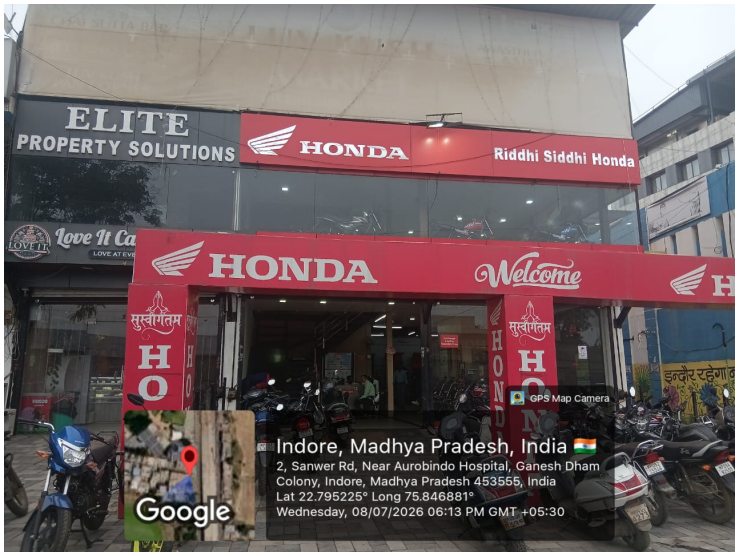


Figure 2: Visit to Riddhi Siddhi Honda Showroom & Service Center

Figure 3: Visit to Riddhi Siddhi Honda Showroom & Service Center

Figure 4: Visit to Riddhi Siddhi Honda Showroom & Service Center

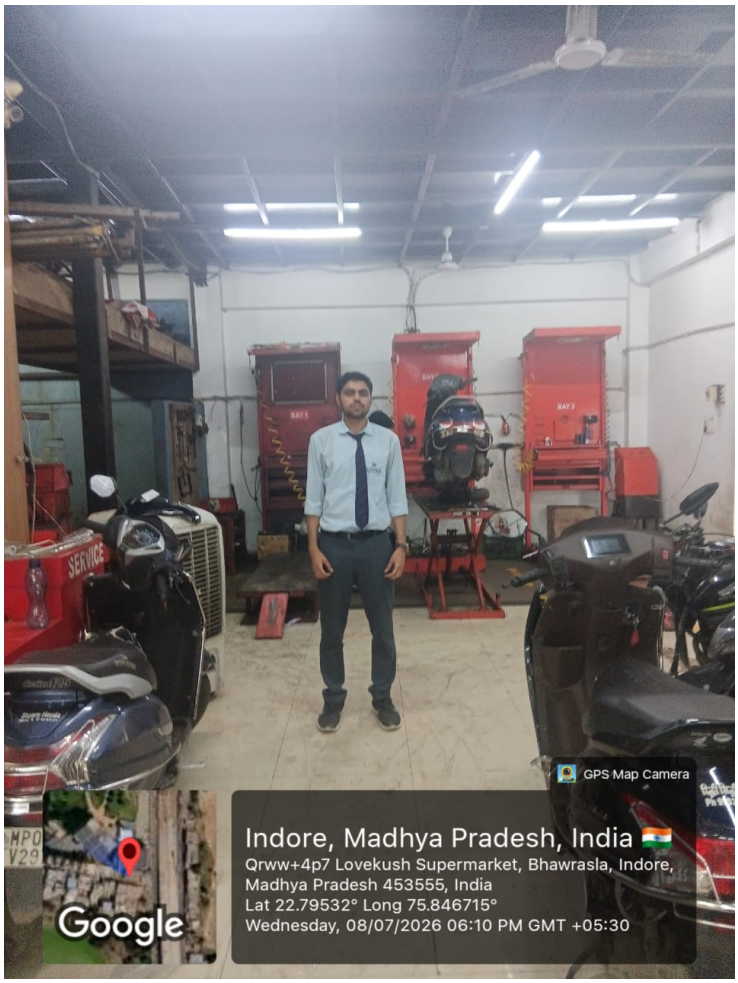


Figure 5: Visit to Riddhi Siddhi Honda Showroom & Service Center

Figure 6: Visit to Riddhi Siddhi Honda Showroom & Service Center

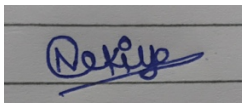
Declaration

We hereby declare that the problems documented in this report are identified through genuine interaction with the officials from the organization and all information is true to the best of our knowledge.

Name: Naman Nekiye

Enrollment No.: 0827CS241159

Signature:



Name: Nikhil Kumar Gautam

Enrollment No.: 0827CS241166

Signature: